GeoDev Lab

a guild of GIS developers in the making

MONTH 2 LEARNING PACK

Development environment and early Python

Weeks 5 to 9, from clicking to writing code

Cohort One

5	21	1	5
WEEKS	DIAGRAMS	REAL PROGRAM	DELIVERABLES

LEARN. BUILD. COLLABORATE. TRANSFORM.

Contents

Before you begin

Why this month gets five weeks · What you will be able to do by the end · How the month builds · The one big idea behind this month · How to use this pack · Five rules that matter this month · What you need installed before Week 5

Deliverables: what you hand in, and how

The one-repository rule · What you hand in each week · Week 5 setup: find your Month 1 repository · How to take the screenshot · How to upload files to GitHub · How to edit your README · Check the link works for a stranger · How to hand in on Google Classroom · What never goes on GitHub · How your work is graded

Week 5: The terminal and VS Code

Two ways to tell a computer what to do · Opening a terminal · Where am I, and what is here · Moving around · Creating things from the terminal · VS Code, where you will write code · Guided practical · Your Week 5 deliverable

Week 6: Environments and project setup with uv

What a package is · The problem environments solve · What uv is, and the three commands · The two files that describe your project · Guided practical · Your Week 6 deliverable

Week 7: Python, the six ideas

The variable · The type · The list · The loop · The condition · The function · Reading an error without panic · Guided practical · Your Week 7 deliverable

Week 8: Python, files and data

print shows a person, return hands the value back · Reading a CSV properly · The dictionary · Reading JSON · Debugging · Handling messy and missing data · Guided practical · Your Week 8 deliverable

Week 9: Consolidation and refactoring

What refactoring is · Two ideas you have already been using · The common errors, in one place · Guided practical · Your Week 9 deliverable and the monthly task

End of Month 2

What you should have built · For the standup · What happens in Month 3 · A word about pace · The one idea to carry forward

For pod leads

Glossary

Before you begin

This is the Month 2 learning pack for GeoDev Lab Africa, Cohort One. It covers Weeks 5 to 9, and it is the single biggest step in the whole programme.

In Month 1 you worked in QGIS, clicking. You chose a spatial question, gathered real Nigerian data, cleaned it and ran your first analysis. All of that was done with your mouse.

This month you stop clicking and start writing. By the end you will have written a real Python program, run it from a terminal, and understood every line of it. It will read the very data you gathered in Month 1 and produce a result.

If you have never programmed in your life, this is the month you learn to. It is written for exactly that person. Nothing is assumed, nothing is skipped, and it moves at the pace of someone meeting each idea for the first time. When a word is new, it is explained where it first appears, and every term is collected in the Glossary at the back.

Why this month gets five weeks

Everything after this month depends on you being able to write and run Python. Month 3 puts your GIS work into code. Later months build a database and a web service in code. Every one of them assumes this month.

So we give it five weeks instead of four, and we go slowly and thoroughly. There is no honest shortcut through learning to program. A week spent properly on what a function is will save you a month of confusion later.

Read that as permission. If this month feels slow, that is correct. Slow and solid beats fast and shaky.

What you will be able to do by the end

- Open a terminal and move around your computer by typing commands
- Set up a professional coding environment, the same one working developers use
- Write Python from scratch: variables, lists, loops, conditions and functions

- Read a file, process every row, and calculate a result
- Read an error message calmly and fix what it points at
- Take a messy working script and refactor it into clean, reusable functions
- Point all of this at your own Month 1 data
- Put your work on GitHub and hand in a link that a grader can open

That is a genuine set of skills. Someone who can do those things is no longer only a GIS analyst. They have started to become a developer.

How the month builds

How Month 2 builds, week by week



You start by clicking in QGIS. You end by writing your own script and understanding every line.

Each week rests on the one before. **Do not skip ahead.** Week 7 assumes the setup from Weeks 5 and 6. Week 8 assumes the Python from Week 7. Week 9 pulls the whole month together. If you jump forward, later weeks will not make sense, and you will think you are failing when really you skipped a step.

The one big idea behind this month

In Month 1 you learned the distinction the whole programme is built on. An analyst runs an analysis once, by hand. A developer builds the system that runs it again and again, without them present.

This month is where you take the first real step across that line. Clicking in QGIS is doing the analysis by hand. Writing a script that does the same thing is building the system. The script runs the same way every time. It runs next month on new data. It runs while you sleep. That is the difference, and this is the month you first make it.

How to use this pack

Each week has the same shape, so you always know where you are.

Preparation notes. Read these first, before you watch anything. They take ten to fifteen minutes and tell you what the week is about.

The teaching sections. Numbered 1, 2, 3 and so on. Each one explains a single idea from nothing, with a diagram and small examples. Read them with VS Code open and try the examples as you go.

Watch the week's video. It is recorded by a working developer and released at the start of each week. It shows the practical being done on a real screen. Watch it, do not skip it.

Do the guided practical. This is the real work. Exact steps, exact commands, and what you should see on your screen when each one works. Do it on your own machine, typing everything yourself.

Try the independent challenge only if the practical felt easy. It is optional and it is harder. Skip it with a clear conscience if you are not ready. It is not graded.

Hand in your deliverable. At the end of every week there is a green box headed "Your Week N deliverable". It lists the exact files to put on GitHub and the link to submit on Google Classroom. The full method is in the Deliverables section straight after this one.

Go to your pod when stuck. After about ninety minutes stuck on the same thing, post in your pod. Not before, because struggle is where learning happens. Not much after, because past ninety minutes you are grinding, not learning.

> *How to read the grey code boxes. Anything in a grey box with a green bar on the left is something you type, or something the computer prints back. When the box follows the words "type" or "run", you type it into the terminal and press Enter. When it follows "create a file" or "type this into the file", you type it into VS Code and save. When it follows "you should see", it is what the computer prints, and you do not type it.*

Five rules that matter this month

Type every command and every line of code yourself. Do not copy and paste. Typing is how your hands and your eyes learn. Copying gives you a working result and teaches you almost nothing. This is the most important rule in the pack. The one exception is the long uv install command, where a single wrong character breaks it.

Read every error out loud before doing anything else. This month you will see many errors. That is not a sign anything is wrong. Errors are how programming works. The whole skill is reading them calmly instead of panicking.

Run your code often. Do not write twenty lines and then run them. Write two lines, run them, confirm they work, write two more. Small steps mean small mistakes that are easy to find.

Keep your Month 1 work close. By Week 8 you will point your new scripts at the data you cleaned in Month 1. This month connects directly to that one. Everything you build goes into the same GitHub repository you started in Month 1. It is one project for the whole twelve months, and it grows every week.

Ask, and answer. Ask a question when you are stuck. Answer somebody else's when you can. Explaining something you half understand is the fastest way to fully understand it.

What you need installed before Week 5

Three free tools. A separate setup guide covers the installation step by step, and there is a video guide that walks through every step on screen: https://youtu.be/VV4p43REQME. Do the installs before Week 5 begins, and give yourself an evening in case something needs a second try.

Python, the language you will write. You may already have it; the setup guide checks before installing anything.

VS Code, a free code editor from Microsoft. Not the same thing as Visual Studio. This is where you will write and run your code.

uv, a small tool that manages your projects. You first use it in Week 6, and the setup guide installs it.

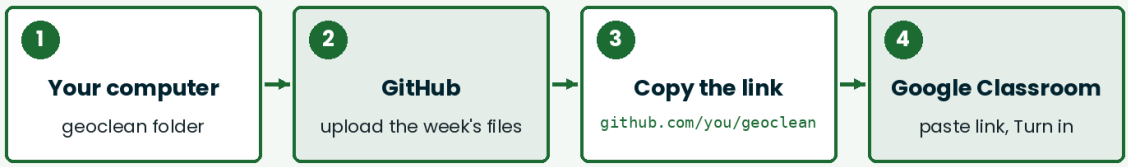
You already have QGIS, a GitHub account and your project repository from Month 1. You keep using all three. Your Month 2 work goes into that same repository.

Deliverables: what you hand in, and how

This section explains exactly what you submit each week, where it goes, and how to hand it in. In short: this week's files go into the same GitHub repository you used in Month 1, and you submit that repository's link on Google Classroom. Read it once now, before Week 5 starts. Come back to it each week when you reach the green deliverable box.

It is written for someone who has never uploaded anything to GitHub. If you already know Git from the terminal, you may use it instead of the website. The result the grader sees is the same.

How your work reaches the grader, every week



GitHub holds your work. Classroom holds the link to it. The grader opens the link.
If the link opens for a stranger in a private browser window, it will open for the grader.

The one-repository rule

A **repository**, often shortened to **repo**, is a project folder that lives on GitHub. It holds your files, and it has a web address anyone can open.

In Month 1 you created a repository for your project. **You keep using that same repository for Month 2, and for every month after it, until Month 12.** You do not create a new repository for Month 2, and you do not create one each week. Your project is one continuous piece of work across the whole year, and your repository is where it lives. By the end of the programme, that one repository shows everything you built, month by month, from your first QGIS map to your final deployed system.

Your Month 1 files stay exactly where they are. This month you add new files alongside them. You never delete or replace your Month 1 work.

> *About the name geoclean. Everybody named their Month 1 repository differently. This pack uses the name geoclean as a stand-in for yours. Wherever you see geoclean, in a folder name, a link or an instruction, use the name of your own Month 1 repository instead. If your repository is called oyo-flood-risk, your link is https://github.com/your-username/oyo-flood-risk and your folder on your computer is called oyo-flood-risk.*

The link you submit on Google Classroom is the address of that repository. It looks like this, with your own GitHub username and your own repository name:

```
https://github.com/your-username/geoclean
```

It is the same link you submitted in Month 1, and you submit it again every week this month. What changes is what is inside the repository. The grader opens your link, looks for the files named for that week, and checks the screenshot. That is why each week adds **new files** rather than changing old ones: your Week 7 file must still be there, unchanged, when Week 9 is graded.

What you hand in each week

Every week has three parts: the week's **files**, one **screenshot** of your terminal proving the code ran on your own computer, and one new line in your **README** saying what you did. The table is the full list. Each week's green box repeats its own row.

Week	Files to upload	Screenshot to upload	What the screenshot must show
5	`hello.py`	`screenshots/week5.png`	`python --version`, `pwd`, and `python hello.py` printing your message
6	`pyproject.toml`, `uv.lock`, `check.py`	`screenshots/week6.png`	`uv --version` and `uv run check.py` printing the pandas version
7	`wards.csv`, `analyse.py`	`screenshots/week7.png`	`uv run analyse.py` classifying each ward as large or small

Week	Files to upload	Screenshot to upload	What the screenshot must show
8	your data CSV, `inspect_data.py`	`screenshots/week8.png`	`uv run inspect_data.py` printing the summary of two of your columns
9	`summary.py`	`screenshots/week9.png`	`uv run summary.py` printing your capstone report

The link you submit, every week: your Month 1 repository link, `https://github.com/your-username/geoclean`, with your own username and repository name.

> *Why a screenshot? Anyone can type a file. The screenshot proves the code actually ran on your machine and produced a result. Make sure your command and its output are both visible in it. A screenshot of code in the editor, without the terminal output, does not count.*

Your Month 1 repository at the end of Month 2: same repo, new files

geoclean/ (your repository name)	
your Month 1 files	stay exactly as they are
README.md	Month 1 content, plus a Month 2 section
hello.py	Week 5
pyproject.toml	Week 6
uv.lock	Week 6
check.py	Week 6
wards.csv	Week 7
analyse.py	Week 7
mydata.csv	Week 8, your Month 1 data (renamed)
inspect_data.py	Week 8
summary.py	Week 9, the capstone
screenshots/	week5.png to week9.png

Week 5 setup: find your Month 1 repository

Do this once, at the start of Week 5, before the guided practical. It takes five minutes.

1. Go to **github.com** and sign in.

2. Click your round profile picture in the top right corner, then click **Your repositories**.
3. Find the repository you used in Month 1 and click its name.
4. Look at the address bar at the top of your browser. It shows your repository link, for example `https://github.com/your-username/geoclean`. **Write down the repository name exactly**, the part after your username. You need it in the Week 5 practical, where you give your folder on your computer the same name.
5. Check it is **Public**: beside the repository name you should see a small label saying **Public**. If it says **Private**, follow the steps in "Check the link works for a stranger" below to change it. A private repository cannot be opened by the grader.

Your Month 1 files are there, and they stay there. Now make one folder on your own computer for screenshots. You do this in Step 4 of the Week 5 guided practical, once your project folder exists, by typing this in the terminal inside it:

```
mkdir screenshots
```

You will save each week's screenshot in there.

> *Your Month 1 repository has files on GitHub, but your Week 5 project folder on your computer starts empty. That is fine. You do not need to download your Month 1 files to your computer to add new ones. GitHub keeps both together: when you upload this week's files, they sit next to your Month 1 files in the same repository.*

If you genuinely do not have a Month 1 repository, because you joined late or it was never created, tell your pod lead first. Then create one: on GitHub click the **+** sign in the top right, then **New repository**. Give it a short name with no spaces that describes your project, choose **Public**, tick **Add a README file**, and click **Create repository**. That is now your repository for the whole year.

How to take the screenshot

Run the week's command in the terminal inside VS Code, so the command and its output are on screen. Make the terminal panel tall enough that nothing important is cut off. Then:

On Windows: press the **Windows key** and **PrtScn** together. The screen dims for a moment. The picture is saved automatically in your **Pictures** folder, inside a folder called **Screenshots**. If your keyboard has no PrtScn key, press **Windows key, Shift and S** together, drag a box around the terminal, then click the notification that appears and use the save icon.

On Mac: press **Command, Shift and 4** together. The cursor becomes a crosshair. Drag a box around the terminal and let go. The picture is saved on your **Desktop**.

Then move the picture into your `geoclean/screenshots` folder and rename it exactly as the table says, for example `week5.png`. On Windows, the extension `.png` is already there if you turned on file extensions in the setup guide; only change the part before it.

How to upload files to GitHub

You upload through the GitHub website. No terminal commands are needed.

Uploading ordinary files (the week's code and data):

1. Open your repository page, `https://github.com/your-username/geoclean`.
2. Near the top right of the file list, click **Add file**, then **Upload files**.
3. Open File Explorer (Windows) or Finder (Mac) and go to your project folder. Select **only the files named for this week**. Drag them onto the GitHub page, into the area that says "Drag files here". You can also click **choose your files** and pick them.
4. Wait until each file appears in the list below with no red error.
5. Scroll down to **Commit changes**. In the first box, write a short message about what you added, for example `Week 7: analyse.py and wards.csv`. A commit is a saved change, and the message is its label.
6. Leave **Commit directly to the main branch** selected. Click the green **Commit changes** button.

The page returns to your repository and the new files are listed. Click one to check it opened properly.

Uploading the screenshot into the screenshots folder:

- **The first time (Week 5):** on the upload page, drag the whole `screenshots` folder from your computer, not just the picture. GitHub keeps the folder, so `screenshots/week5.png` appears in your repository.

- **Every week after:** on your repository page, click the `screenshots` folder to open it first. Then click **Add file**, **Upload files**, and drag in that week's picture. It lands inside the folder.

> *Never drag the whole geoclean folder onto GitHub. From Week 6 it contains a hidden folder called .venv with thousands of files. GitHub's website only accepts 100 files at a time, so the upload fails, and those files do not belong on GitHub anyway. Always select the week's files by name.*

If you uploaded the wrong file, click it on GitHub, click the three dots (**…**) at the top right of the file, choose **Delete file**, and click **Commit changes**. **To replace a file** with a newer version, upload it again with exactly the same name; GitHub keeps the new one.

How to edit your README

The `README.md` file is the front page of your repository. GitHub shows it under your file list. It tells the grader, and later an employer, who you are and what the project does.

To edit it on the website:

1. On your repository page, click `README.md`.
2. Click the pencil icon near the top right of the file.
3. Type your changes in the box.
4. Click **Commit changes**, write a short message, and click **Commit changes** again.

Your README already has your Month 1 content. **Do not delete it.** In Week 5, scroll to the bottom of the README and add a new section for this month. Type it like this, with your own details:

```
## Month 2: development environment and early Python

- Week 5: set up Python, VS Code and the terminal. hello.py runs.
```

The `##` makes a heading and the `-` makes a list item; GitHub turns them into neat formatting. If your README does not yet say who you are, add your full name and pod near the top as well.

Then each week, add one line under your Month 2 heading, such as `- Week 7: analyse.py reads wards.csv and classifies each ward.` In Week 9 you add a short

reflection, described in the Week 9 deliverable. Month 3 will get its own heading underneath, and so on through the year, so your README becomes a record of the whole project.

> *From Week 6, uv also makes a README.md inside your geoclean folder on your computer. Do not upload that one. It would replace the README you have been writing on GitHub since Month 1. Your README lives on GitHub; edit it there.*

Check the link works for a stranger

Before you hand in, prove the grader can open your link. Open a **private window** in your browser: in Chrome or Edge press **Ctrl, Shift and N** (Command, Shift and N on Mac); in Safari or Firefox use Command or Ctrl, Shift and P. Paste your repository link into that window.

If you see your files, the link works. If you see a **404** page ("This is not the web page you are looking for"), your repository is private or the link has a typo. To make it public: on the repository page click **Settings**, scroll to the bottom to **Danger Zone**, click **Change visibility**, choose **Public**, and confirm.

How to hand in on Google Classroom

1. Go to **classroom.google.com** and sign in with the email address you used to join the GeoDev Lab Africa class. If you see the wrong account, click your picture in the top right and switch.
2. Open the class, then click the **Classwork** tab at the top.
3. Click the task for the week, for example the Month 2 Week 7 task, then click **View instructions** or **View assignment**.
4. On the right, in the **Your work** panel, click **+ Add or create**, then choose **Link**.
5. Paste your repository link, the same Month 1 link, `https://github.com/your-username/geoclean`, and click **Add link**.
6. Click **Turn in**, then **Turn in** again to confirm. The status changes to **Turned in**.

Make sure you open the Month 2 task for the week, not an old Month 1 task. Upload to GitHub **first**, then turn in. The grader looks at your repository as it is when they open it, so the week's files and screenshot must already be there. Hand in by the deadline shown on the task in Classroom.

If you need to fix something after turning in, click **Unsubmit** on the task, make the change on GitHub, and click **Turn in** again. If something prevents you submitting on time, use the **private comment** box on the task to tell the grader why, before the deadline.

What never goes on GitHub

- **The .venv folder.** It is rebuilt from `uv.lock` in seconds on any computer.
- **Passwords, keys and tokens** of any kind, even inside a code file.
- **Personal data about real people** that you do not have the right to share, such as names, phone numbers or addresses in a survey. If your Month 1 data contains this, remove those columns before uploading, or upload a copy with only the columns your script uses.
- **Very large files.** GitHub's website refuses a file over 25 MB. If your Month 1 CSV is larger, upload a smaller copy containing the first few thousand rows, and say so in your README.

How your work is graded

Weekly tasks count for 30 percent of your gradebook, weekly quizzes for another 30 percent, and monthly tasks for 40 percent. To stay in good standing you need 70 percent completion across the programme, and at least 50 percent in any single month. Quizzes allow a single attempt.

A weekly task is graded on five plain questions. Are the week's files in the repository with the exact names given? Does the screenshot show the command and its output? Does the code do what the practical asked? Are your Month 1 files still there? Is there a new README line? Nothing is graded on how elegant the code is. Honest, working, and yours is what counts.

Week 5: The terminal and VS Code

Your milestone this week: a working development environment on your own machine, you can move around your computer by typing commands in the terminal, and your first Month 2 file sits in your Month 1 repository on GitHub.

Preparation notes

Read this before the video. It takes ten minutes and it will make the video make sense.

This week is about two things: the terminal, which is the plain window that puts a lot of people off, and VS Code, which is where you will write code. Neither is difficult. The terminal has perhaps twenty commands you actually use day to day, and by the end of this week you will know the six that matter most.

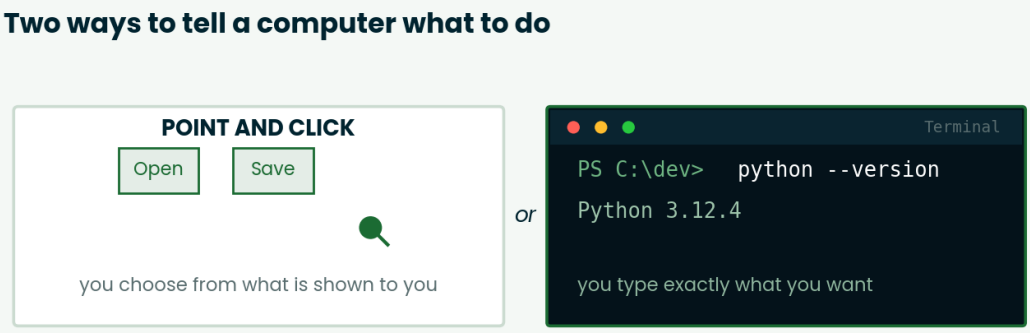
You will not write real Python this week. That starts in Week 7. This week is about getting comfortable in the tools, so that when the real coding starts you are not fighting the environment at the same time.

Have Python and VS Code installed before you start, using the setup guide. If an install went wrong, bring the exact error message to your pod. “It did not work” cannot be helped; the exact words of the error can be fixed in one reply.

What this week covers

Why developers use the terminal at all, the handful of commands that let you move around your computer, setting up VS Code so it understands Python, and adding this month’s work to the GitHub repository you started in Month 1.

1. Two ways to tell a computer what to do



The terminal looks harder and is often faster. The real reason developers use it:
a command can be written down and repeated exactly. A click cannot.

Everything you have ever done on a computer, you have probably done by pointing and clicking. You click a button, you choose from a menu, you drag a file. The computer shows you the options, and you pick one. This is called a **graphical interface**, and it is genuinely good. QGIS is a graphical interface. So is your phone.

The **terminal** is the other way. Instead of clicking, you type an instruction, and the computer carries it out. The instruction is called a **command**.

Here is the honest comparison. For many everyday tasks, the terminal is faster once you know it. But that is not the main reason developers live in it. The main reason is this:

A command can be written down and repeated exactly. A click cannot.

Think about what that means. You cannot email somebody your mouse movements. You cannot write down “click the third button, then the menu, then the second option” in a way a computer can run. But you can write down a command, save it, share it, and run it a thousand times, and it does exactly the same thing every time.

That is the whole reason. Everything in software that has to be repeatable, that has to run without a person clicking, is built on typed commands. From here to the end of the programme, installing tools, running your code and publishing your work all happen by typing commands. This week you meet the terminal properly so that none of that is frightening later.

2. Opening a terminal

The terminal is a program like any other. Here is how to open one.

On Windows. Press the Windows key, type `powershell`, and press Enter. PowerShell is the terminal that comes with Windows. A window opens with some text and a blinking cursor. If you see a program called **Windows Terminal** instead, that is fine too; it opens PowerShell inside it.

On Mac. Press Command and Space together to open Spotlight, type `terminal`, and press Enter. A window opens with a prompt.

On Linux. You almost certainly know where your terminal is. Ctrl, Alt and T opens one on most systems.

When it opens you will see a line ending in a symbol like `>` or `$` or `%`, with a cursor blinking after it. That symbol is called the **prompt**. It is the terminal saying "I am ready, type something". The text before the symbol is usually the folder you are in, which section 3 explains.

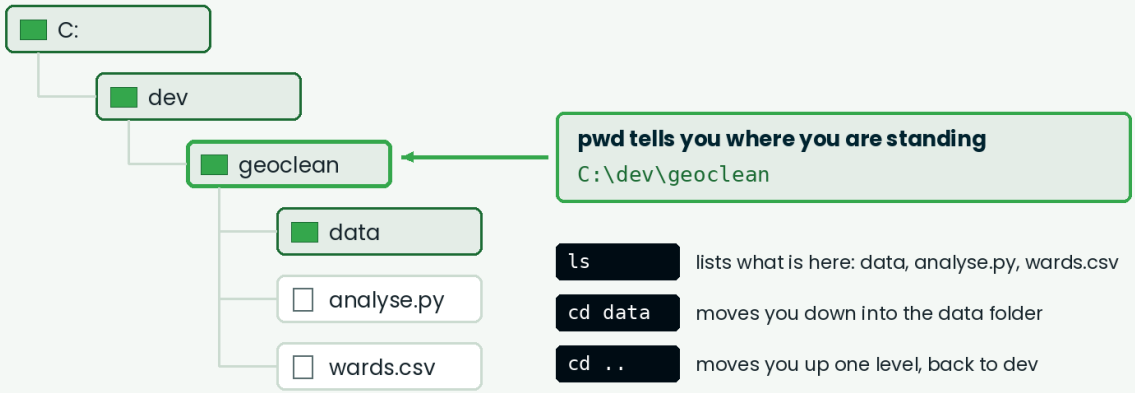
Everything you type goes after the prompt. You type a command, press Enter, and the terminal runs it and shows you the result. Then it gives you a fresh prompt, ready for the next command.

> *Two things that surprise beginners. First, the terminal often prints nothing when a command succeeds. Silence usually means it worked. Second, when this pack shows a prompt such as PS C:> before a command, you do not type the prompt. You type only the command after it.*

3. Where am I, and what is here

The single most important idea about the terminal is this: **the terminal is always sitting inside one folder on your computer.** It has a location. Every command you run happens in that location unless you say otherwise.

The filesystem is a tree, and you are always standing somewhere in it



Same files as your file explorer. The terminal just speaks to them in words.

Your computer's files are arranged in a tree. At the top is the drive, `C:` on Windows or `/` on Mac. Inside it are folders, and inside those are more folders and files. "Folder" and "directory" mean exactly the same thing; developers tend to say directory. The terminal is always standing at one point in that tree, and two commands tell you about where you are standing.

`pwd` stands for "print working directory". It prints the folder you are currently in. When you feel lost, and you will, this is the first thing to type.

Type it and press Enter:

```
pwd
```

On Windows PowerShell you will see a small heading, `Path`, a line of dashes, and then something like `C:\Users\Temitope`. On Mac you will see something like `/Users/temitope`. That is your current location, usually called your **home folder**.

`ls` lists what is in the current folder. On Windows, `dir` does the same thing, but `ls` works in both Windows PowerShell and on Mac, so learn `ls` and use it everywhere.

```
ls
```

You will see a list of the files and folders in your current location. On Windows, folders are marked with a `d` in the first column; on Mac they are simply listed by name.

Here is the thing that removes the fear entirely. Open your normal file explorer, Windows File Explorer or Mac Finder, and go to the same folder that `pwd` showed you. You are now looking at the same folder two different ways: the graphical window, and

the terminal. There is nothing hidden or magic happening in the terminal. It is the same computer, the same files, just spoken to with words instead of clicks.

4. Moving around

You move through the tree of folders with one command: `cd`, which stands for "change directory". Always put a space between `cd` and the folder name.

Go into a folder by naming it:

```
cd Documents
```

If there is a folder called Documents where you are standing, this moves you into it. Run `pwd` again and you will see your location has changed.

Go up one level, back to the folder above, with two dots:

```
cd ..
```

The two dots mean "the folder that contains this one". This is how you back out. Run `pwd` to confirm you moved up.

Go straight to a specific place by giving the full path:

```
cd C:\Users\Temitope\dev
```

A **path** is the full address of a folder, written from the top of the tree down. On Windows the parts are separated by backslashes, `\`. On Mac they are separated by forward slashes, `/`, so the same idea looks like `cd /Users/temitope/dev`.

> *Avoid spaces in folder and file names. A folder called My Project confuses the terminal, which reads the space as the end of the name. Use geoclean, my_project or month1-data instead. If you must enter a name that has a space, wrap it in quotes: cd "My Project".*

Now the two habits that make the terminal bearable. These matter more than any single command.

Tab completion. Start typing a folder or file name, just the first few letters, then press the Tab key. The terminal finishes the name for you. If you are in a folder with Documents in it, type `cd Doc` and press Tab, and it becomes `cd Documents`. You almost

never type a full name again. As a bonus, if Tab does not complete something, that thing does not exist where you are, which is useful to know instantly.

The up arrow. Press the up arrow key and your previous command comes back. Press it again for the one before that. You almost never retype a command. You press up, maybe edit it, and run it again.

Spend real time getting comfortable with Tab and the up arrow. They are the difference between the terminal feeling like a chore and feeling like a tool.

5. Creating things from the terminal

You can make folders and files by typing, not just by right-clicking.

Make a folder with `mkdir`, which is short for "make directory":

```
mkdir myproject
```

That creates a folder called `myproject` where you are standing. Run `ls` and you will see it appear.

Make an empty file. This is one of the few places Windows and Mac differ.

On Windows PowerShell:

```
ni notes.txt
```

On Mac or Linux:

```
touch notes.txt
```

Both create an empty file called `notes.txt`. `ni` is short for "new item".

You do not need many more commands than these to get through this month. `pwd`, `ls`, `cd`, `mkdir`, and the way to make a file. Here they are in one place, to keep beside you this week:

Command	What it does	Example
`pwd`	Shows the folder you are standing in	`pwd`
`ls`	Lists what is in this folder	`ls`
`cd name`	Moves into a folder	`cd geoclean`

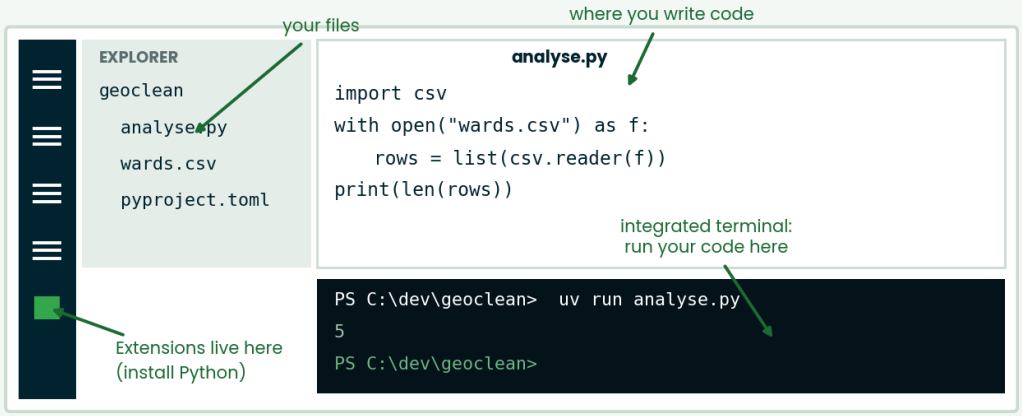
Command	What it does	Example
`cd ..`	Moves up one level	`cd ..`
`mkdir name`	Makes a new folder here	`mkdir data`
`ni name` (Windows) / `touch name` (Mac)	Makes an empty file here	`ni notes.txt`
`clear`	Wipes the screen clean (nothing is deleted)	`clear`
Tab key	Finishes a name you started typing	`cd geo` then Tab
Up arrow	Brings back your last command	press it

6. VS Code, where you will write code

The terminal is for running things. VS Code is for writing them. It is a **code editor**: a program designed for writing code, the way QGIS is designed for maps. It is like a very capable notepad that understands programming languages.

You will spend a great deal of the next year in VS Code, so this week sets it up properly. There are only a few things you need it to do.

The VS Code window, and the four parts you use



Four parts of the window matter, and the rest you can ignore for now.

The activity bar runs down the far left, a strip of icons. The top icon, two overlapping pages, is the Explorer. The one that matters this week is **Extensions**, which looks like four squares. That is where you add the Python extension.

The explorer shows the files in the folder you have open. This is your project, laid out so you can click between files.

The editor is the large central area where you actually write code. When you open a file, its contents appear here. Each open file gets a tab along the top.

The integrated terminal is a terminal built into VS Code, at the bottom. This is important: it means you can write your code in the editor and run it in the terminal without leaving the window. Editor above, terminal below, one program. It is the same PowerShell or Terminal you opened before, just placed inside VS Code.

The single most important addition is the **Python extension**, published by Microsoft. It teaches VS Code to understand Python: it colours your code so it is readable, it points out mistakes before you even run anything, and it helps run your programs. You install it once and forget about it.

> *Is my file saved? Look at the file's tab at the top of the editor. A white dot on the tab means there are unsaved changes; an X means it is saved. If Auto Save is on (File, Auto Save, from the setup guide) you rarely need to think about it. If it is off, press Ctrl+S (Cmd+S on Mac) before every run. Running an unsaved file runs the old version, which is one of the most common beginner puzzles.*

Guided practical

Outcome: a working setup, a project folder you can navigate from the terminal, VS Code understanding Python, and your first file on GitHub.

Work through these in order, on your own machine, typing everything yourself. Do not paste. If a step does not produce what is described, stop and read the "When it breaks" table before moving on.

Step 1: Open a terminal

Open PowerShell on Windows, or Terminal on Mac, using the instructions in section 2. You should see a prompt with a blinking cursor.

Step 2: Check whether Python is installed

Type exactly this and press Enter:

```
python --version
```

If you see something like `Python 3.12.4`, and the middle number is 11 or higher, Python is installed and you are fine. Move to Step 3.

On Mac, if that command says "command not found", try:

```
python3 --version
```

Mac often uses `python3` rather than `python`. If that shows a version, use `python3` wherever this pack says `python`. This applies only to plain `python` commands; commands starting with `uv` are the same on every computer.

If neither works, if the Microsoft Store opens, or the version is 3.10 or lower, install Python using the setup guide, ticking "Add python.exe to PATH" on Windows. Then close the terminal, open a brand new one, and check again.

Step 3: See where you are

```
pwd
```

Read the location it prints. That is where you are standing, your home folder.

```
ls
```

Look at what is listed. Now open your file explorer to that same location and confirm you see the same things. This is the moment the terminal stops being mysterious.

Step 4: Make your project folder

First, do the Week 5 setup in the Deliverables section if you have not already, so you know your Month 1 repository's exact name.

You are going to create a folder called `dev` in your home folder to hold your coding work, and inside it a folder for your project, **with exactly the same name as your Month 1 repository**. Matching names keep things simple: one project, one name, on GitHub and on your computer. If you already have a `dev` folder, skip the `mkdir dev` line.

```
mkdir dev
cd dev
```

Type these one line at a time, pressing Enter after each. Then, inside `dev`, make your project folder and go into it. The example uses `geoclean`; type your own repository name instead:

```
mkdir geoclean
cd geoclean
```

Run `pwd` to confirm you are now inside your project folder. The path will be longer than the diagram showed, something like `C:\Users\Temitope\dev\geoclean` or `/Users/temitope/dev/geoclean`. Write your path down; you will come back to it every time you work, this month and for the rest of the year.

While you are here, make the screenshots folder:

```
mkdir screenshots
```

Step 5: Open the folder in VS Code

If VS Code installed its command line helper, you can open the current folder in VS Code by typing:

```
code .
```

There is a space between `code` and the dot. The dot means "this folder, the one I am standing in". VS Code opens with your `geoclean` folder loaded in the explorer. If VS Code asks **"Do you trust the authors of the files in this folder?"**, click **Yes, I trust the authors**. It is your own folder.

If `code .` is not recognised, open VS Code the normal way, then use **File**, **Open Folder** (on Mac, **File**, **Open**), and choose your `geoclean` folder. To make the shortcut work next time: inside VS Code, press Ctrl+Shift+P (Cmd+Shift+P on Mac), type `shell command`, and choose "Install 'code' command in PATH".

Step 6: Install the Python extension

Skip this if you did it in the setup guide.

1. In VS Code, click the Extensions icon in the activity bar on the left, the four squares.
2. In the search box at the top, type `Python`.
3. The first result should be **Python**, published by **Microsoft**. Click it, then click the blue **Install** button.

4. Wait for it to finish. That is all.

Step 7: Open a terminal inside VS Code

From the top menu, choose **Terminal**, then **New Terminal**. On Windows you can also press Ctrl and the backtick key (the key above Tab) together.

A terminal panel opens at the bottom of the VS Code window, and it is already sitting inside your `geoclean` folder. From now on, this is where you work. You write code in the editor above, and run it in this terminal below, without ever leaving VS Code.

Confirm it by typing `pwd` in this new terminal. It should show your `geoclean` folder.

Step 8: Prove it all works

Create a tiny file to confirm the whole setup runs. In the VS Code explorer, hover over your project's name at the top of the explorer (it appears in capitals) and click the **New File** icon (a page with a plus sign), or use **File**, **New File**. Name it `hello.py` and press Enter. The `.py` ending tells the computer it is a Python file; without it nothing works. Type this one line into it, do not paste it:

```
print("My setup works")
```

The quotes are straight double quotes, typed with Shift and the key next to Enter. Save with Ctrl+S (Cmd+S on Mac). Then, in the integrated terminal at the bottom, type:

```
python hello.py
```

If you see `My setup works` printed in the terminal, everything is working: Python runs, VS Code saved your file, and the terminal found it. That is the whole environment, proven.

> *VS Code shows a small triangle play button at the top right of Python files. Please do not use it this month. It sometimes runs a different Python from the one you expect, which causes confusing errors from Week 6 onwards. Always run your code by typing the command in the terminal. That is also what the screenshots ask for.*

Step 9: Hand in

Now add this week's work to your Month 1 repository, following the Deliverables section and the green box below. You are not creating a new repository. Your `hello.py` and screenshot go in next to your Month 1 files.

You are done when

- `python --version` (or `python3 --version` on Mac) shows a version 3.11 or higher
- You can use `pwd`, `ls` and `cd` to move around and see where you are
- VS Code opens your `geoclean` folder
- The Python extension by Microsoft is installed
- A terminal inside VS Code shows your project folder
- Running `python hello.py` prints your message
- `hello.py` and your screenshot are in your Month 1 repository, and your deliverable is turned in

Independent challenge

Only if the practical above felt comfortable. Not graded.

Explore three more commands. Find out what each of these does by trying it on your `notes.txt` or `hello.py` file, and write a one-line note on each: `cat` shows a file's contents (it works in PowerShell too, where `type` also works); `cp` copies a file, as in `cp hello.py hello_copy.py`; `mv` moves or renames one, as in `mv hello_copy.py renamed.py`. Test them, and confirm with `ls` that they did what you expected.

Then, harder. Using only the terminal, without touching the file explorer, create a folder called `practice` inside `geoclean`, copy `hello.py` into it with `cp hello.py practice`, and then list the contents of `practice` to confirm it arrived. This is pure terminal navigation, which you will do constantly from now on. Keep your original `hello.py` where it is, because it is your deliverable.

When it breaks

What you see	What it means	What to do
`python` is not recognised	Python is not installed, PATH was not ticked, or the terminal is old	Install from the setup guide ticking "Add to PATH", then open a fresh terminal
Typing `python` opens the Microsoft Store	Real Python is not installed	Install from python.org using the setup guide
`python --version` shows 2.7	An old Python 2 is first in line	Use `python3` instead, or install a current version

What you see	What it means	What to do
On Mac, `python` not found	Mac uses `python3`	Use `python3` everywhere this pack says `python`
`code .` is not recognised	The VS Code shell command was not installed	In VS Code: Ctrl+Shift+P, then "Install 'code' command in PATH"
`cd` says it cannot find the path	The folder does not exist where you are, or there is a typo	Run `ls` to see what is really here, and use Tab to complete the name
`python hello.py` says "No such file or directory"	The terminal is in a different folder from the file, or the file is not named `hello.py`	Run `pwd` and `ls`. On Windows, check the file is not really `hello.py.txt`
Nothing happens when I run the file	You ran it before saving	Save with Ctrl+S, or turn on Auto Save, then run again
The terminal opens in the wrong folder	You opened it outside VS Code, or before navigating	`cd` into your project, or use the terminal inside VS Code
My code has no colours in VS Code	The Python extension is not installed, or the file does not end in `.py`	Install the Microsoft Python extension; rename the file to end in `.py`
I cannot find PowerShell	Unusual or older Windows	Search for "Terminal" or "Command Prompt"; the commands in this pack work the same
`mkdir` says the folder already exists	You already made it	That is fine, just `cd` into it

Check yourself

Answer these out loud, without looking back.

Why do developers use the terminal rather than only clicking? Because a command can be written down and repeated exactly, which a click cannot. Anything that must be repeatable is built on typed commands.

What does `pwd` tell you? The folder you are currently standing in.

What does `cd ..` do? Moves you up one level, to the folder that contains the current one.

What do Tab and the up arrow do? Tab completes a name you have started typing. The up arrow brings back your previous command. Together they mean you rarely type anything in full.

What is the Python extension for? It teaches VS Code to understand Python, so it colours your code, warns about mistakes, and helps run your programs.

Your Week 5 deliverable

> **Upload to your Month 1 repository on GitHub:** `hello.py`, and the `screenshots` folder containing `week5.png`. Do not create a new repository.
>
> **The screenshot shows** the VS Code terminal after you run these three commands, one after another: `python --version`, then `pwd`, then `python hello.py`. All three commands and their output must be visible.
>
> **Add to your README:** keep your Month 1 content, and add the Month 2 heading and the Week 5 log line at the bottom, as shown in the Deliverables section.
>
> **Submit on Google Classroom,** on the Month 2 Week 5 task: your Month 1 repository link, `https://github.com/your-username/geoclean`, then click Turn in.

Week 6: Environments and project setup with uv

Your milestone this week: a reproducible Python project with its own environment, a package installed, the setup recorded in two files, and those files on GitHub.

Preparation notes

Read this before the video. Ten minutes.

This week has one main idea, and it is not obvious at first: why a project should have its own private, separate space for the tools it uses. Once you see the problem this solves, the how is just three commands.

The tool you will use is called **uv**. There are older tools that do similar jobs, and you will hear about them, but uv is what this programme uses because it is fast, modern and simple. You will learn one clear path rather than juggle several.

Have your `geoclean` folder from Week 5 open in VS Code, with the integrated terminal open at the bottom.

What this week covers

What a package is, why every project needs its own environment, what uv is and the three commands you use, and the two files that let anyone rebuild your project exactly.

1. What a package is

Before environments, you need to understand packages, because environments exist to manage them.

A package is code somebody else wrote, that you can use



You install a package once with uv add pandas. From then on, import brings it into your code.

There are hundreds of thousands of packages. A big part of the job is knowing which to reach for.

When you write Python, you do not write everything from scratch. Somebody, somewhere, has already written excellent code for reading spreadsheets, for working with dates, for talking to the internet, for handling maps. They have packaged that code up and shared it, so that you can use it without writing it yourself. That bundle of shared code is called a **package**. Some people call it a library; for this month the two words mean the same.

A package you will meet constantly is **pandas**, which handles tables of data. Somebody spent years making pandas work well. You get to type `import pandas` and use all of it.

You bring a package into your code with the word `import`:

```
import pandas
```

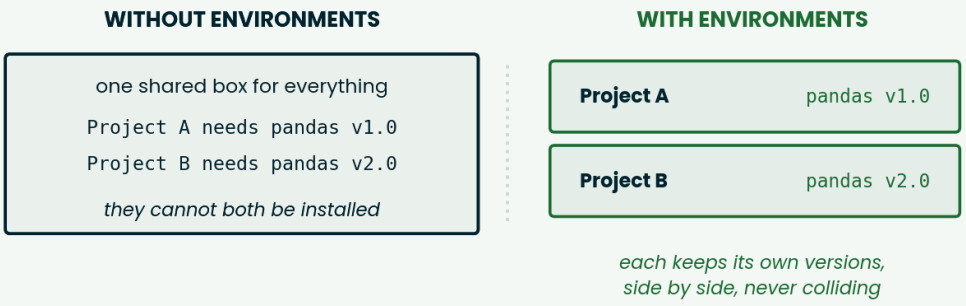
After that line, everything pandas can do is available to your program.

Some packages come built into Python, such as `csv` and `json`, which you use in Week 8. Those you can import straight away. Others, like pandas, must be **installed** onto your computer first, from a free public collection called PyPI, the Python Package Index. There are hundreds of thousands of them. A real part of becoming a developer is not memorising packages, but knowing that they exist and which one to reach for. Nobody remembers them all. Everybody looks them up.

But installing packages is where a problem appears, which is what environments solve.

2. The problem environments solve

Why each project gets its own environment



uv creates and manages these boxes for you. One per project.

pyproject.toml records what each project needs, so anyone can rebuild it exactly.

Imagine you have two projects. One you built a year ago, and it needs an old version of pandas, version one, because that is what it was written for. The other is new, and needs the latest version, version two, because it uses a new feature.

Now, if every package installs into one single shared place on your computer, those two versions fight. You can only have one version of pandas installed at a time in a shared place. Installing version two for your new project breaks the old one. Installing version one for the old project breaks the new one. You are stuck.

An environment is a private box of installed packages, belonging to one project.

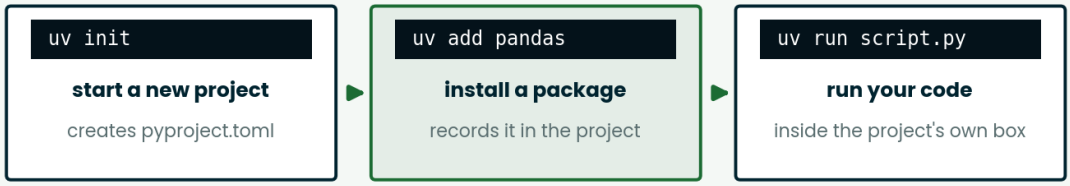
Project A gets its own box, with its own version of pandas. Project B gets a separate box, with a different version. They never touch each other. Each project has exactly what it needs, and nothing interferes. On your computer, the box is simply a hidden folder called `.venv` inside your project folder. The name is short for "virtual environment", and the dot at the start is what makes it hidden.

This is not an advanced technique or an optional nicety. Every serious Python project has its own environment. It is simply how the language is meant to be used, and the tools make it easy.

3. What uv is, and the three commands

`uv` is a tool that creates and manages these environments for you, and installs packages into them. It does three things you use all the time.

The three uv commands you use all the time



That is the whole daily workflow. Start a project, add what you need, run it.

`uv init` starts a new project. You run it once, in your project folder, and it sets up the files a project needs.

`uv add` installs a package into your project's environment, and records that you did. For example, `uv add pandas` installs pandas into this project's private box.

`uv run` runs your code inside the project's environment, so your code can see the packages you added. For example, `uv run analyse.py` runs your script with pandas available.

That is the whole daily rhythm. Start a project, add what you need, run your code. Three commands. From this week on, you run your Python files with `uv run`, not plain `python`.

A brief note on pip. You may have heard of `pip`, an older tool for installing packages, and you will see it in many tutorials online. It works, but it does not manage environments for you as cleanly, and it is easy for a beginner to install packages into the wrong place with it. This programme uses `uv` instead, which handles the environment automatically. If you see `pip install something` in a tutorial elsewhere, the uv equivalent is `uv add something`. Knowing this means you will not be confused when you meet `pip` in the wild.

4. The two files that describe your project

When you use `uv`, two files appear in your project. It is worth understanding what each does, because they are how your work becomes shareable.

The two files that describe your project



Both go into version control. Together they are the exact recipe for rebuilding your project anywhere.

`pyproject.toml` is the human-readable description of your project. It holds your project's name, the Python version it needs, and the list of packages it depends on. This is a file you can open and read, and it will make sense. When you run `uv add pandas`, this file is updated to record that your project needs pandas. You will rarely edit it by hand, because uv edits it for you, but you can read it any time to see what your project depends on. The odd ending, `.toml`, is just the name of a simple settings format.

`uv.lock` is the exact, precise record. Not just "this project needs pandas", but "this project needs pandas version 2.2.1 exactly, which itself needs numpy version 1.26.4 exactly, which needs…", all the way down. It lists every package, and every package those packages depend on, pinned to an exact version. You never edit this file. uv writes it and maintains it. (The version numbers in the diagram are examples. Yours will be whatever is current when you install.)

Why does the second file matter? Because it is what makes your project **reproducible**. When you put your project on GitHub, somebody else, or you on a different computer, can download it, run one command, and get an environment identical to yours. Same packages, same versions, exactly. Nothing is left to chance.

That is the difference between a script that "works on my machine" and software that works anywhere. It is one of the things that separates a hobby from a profession, and uv gives it to you almost for free. That is also why these two files are this week's deliverable, while the `.venv` folder never goes to GitHub: the recipe travels, and the box is rebuilt from it.

Guided practical

Outcome: your `geoclean` project has its own environment, a package installed, and a script that uses it, all recorded so anyone could rebuild it.

Step 1: Install uv

If you installed uv from the setup guide, skip to Step 2. Otherwise, in your terminal:

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 |
iex"
```

Mac or Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Then close the terminal, open a fresh one, and check it worked:

```
uv --version
```

You should see a version number. If you see "not recognised", you opened the terminal before the install finished; close it and open another. If you are using the terminal inside VS Code, close VS Code completely and reopen it, because the VS Code terminal also needs a fresh start. This fresh-terminal step is the same lesson from Week 5, and it catches everybody at least once.

Step 2: Go into your project

Make sure you are inside your `geoclean` folder. Check with:

```
pwd
```

It should end in `geoclean`. If not, `cd` your way there, for example `cd dev` then `cd geoclean` from your home folder.

Step 3: Start the project

```
uv init
```

This sets up your project. You should see a message such as `Initialized project geoclean`. Run `ls` and you will see new files appear:

- `pyproject.toml`, the project description

- `main.py`, a tiny example program uv writes for you. You can leave it alone.
- `README.md`, a local readme. Leave it alone and do not upload it; your real README lives on GitHub.
- `.python-version` and `.gitignore`, two small settings files. The dot makes them hidden in Finder, which is normal.

Your `hello.py` from Week 5 is untouched. uv has turned your folder into a proper Python project, alongside what was already there.

Step 4: Look at what it made

Click `pyproject.toml` in the VS Code explorer to open it. It looks like this:

```
[project]
name = "geoclean"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12"
dependencies = []
```

Read it. The `requires-python` number may differ on your machine. The line `dependencies = []` is an empty list: your project does not depend on any packages yet. Do not change anything; just see that it is a plain, understandable description of your project.

Step 5: Add your first package

Add pandas, which you will use for the rest of the year:

```
uv add pandas
```

Watch what happens in the terminal. uv creates the environment for this project, the hidden `.venv` folder, installs pandas and the packages pandas needs into it, and updates both `pyproject.toml` and `uv.lock` to record it. The first time can take a minute and needs internet. Later installs are much faster.

If VS Code shows a small box saying it noticed a new environment and asking whether to select it, click **Yes**.

Look at `pyproject.toml` again. The dependencies line now contains pandas, something like `"pandas>=2.2.1"`. uv added it for you. A file called `uv.lock` has also appeared.

Step 6: Write a script that uses the package

In VS Code, create a new file called `check.py` and type this in, do not paste:

```
import pandas

print("pandas version:", pandas.__version__)
```

`pandas.__version__` asks pandas which version it is. There are two underscores on each side of `version`. This little script proves pandas is installed and your code can reach it.

Step 7: Run it inside the environment

Run it with `uv run`, which uses your project's environment:

```
uv run check.py
```

You should see something like `pandas version: 2.2.3` printed. That output proves pandas is installed in this project's own box, and your code can use it.

If you instead type plain `python check.py`, you will probably get `ModuleNotFoundError: No module named 'pandas'`. That is not a broken install. Plain `python` looks outside the box, where pandas is not installed. `uv run` is what makes the environment's packages available. Try it once, see the error, and you will understand environments better than any explanation.

Step 8: Hand in

Your project now has `pyproject.toml`, `uv.lock` and `check.py`. These are your project's recipe and your first code that uses a package. Take the Week 6 screenshot and upload the files as described in the deliverable box below.

> *If you are comfortable with Git already, you may commit and push from the terminal instead of uploading on the website. Git from the terminal is taught properly later in the programme. Until then, the GitHub website is all you need, and nobody is marked down for using it.*

You are done when

- `uv --version` shows a version
- `uv init` created `pyproject.toml` in your `geoclean` folder
- `uv add pandas` installed pandas and it now appears in `pyproject.toml`
- `uv run check.py` printed the pandas version
- Your Week 6 deliverable is on GitHub and turned in

Independent challenge

Only if the practical felt easy. Not graded.

Add a second package and read the lock file. Run `uv add requests`, a package for talking to the internet. Then open `uv.lock` and scroll through it. You will find far more than two packages listed, because pandas and requests each depend on other packages, which depend on others still. Write one sentence in your notes explaining why the lock file is so much longer than your list of two packages. If you do this, upload the updated `pyproject.toml` and `uv.lock` again so GitHub has the current versions.

Then, harder. Delete the hidden environment folder, `.venv`, from inside your project. In the terminal, on Windows: `Remove-Item -Recurse -Force .venv`. On Mac: `rm -rf .venv`. Type these very carefully, because they delete without asking. Then run `uv run check.py` again. Watch uv rebuild the entire environment from `uv.lock` in seconds, and your script run as if nothing happened. That is reproducibility in action: the recipe was enough to recreate everything.

When it breaks

What you see	What it means	What to do
`uv` is not recognised	Installed, but the terminal was open before it finished	Close the terminal, or close and reopen VS Code
The install command fails on Windows	The execution policy is blocking scripts	Use the exact command in Step 1, which sets a bypass
`uv init` says the project is already initialised	There is already a `pyproject.toml` here	That is fine. Skip to Step 5
`uv add pandas` fails with a network error	No internet, or a very slow connection	Check your connection and try again. It only downloads once
`uv run check.py` cannot find the file	Wrong folder, or the filename is mistyped	Run `ls` to confirm `check.py` is here and spelled correctly
`ModuleNotFoundError: No module named 'pandas'`	pandas was not added, or you ran with plain `python`	Run `uv add pandas`, then use `uv run`, not `python`

What you see	What it means	What to do
`import pandas` has a wavy yellow line in VS Code	VS Code is looking at a different Python from your project's	Press Ctrl+Shift+P, type `Python: Select Interpreter`, choose the one with `.venv` in its path. The code runs fine either way with `uv run`
`AttributeError` about `__version__`	Only one underscore on each side	Type two underscores before and after `version`
I do not understand `pyproject.toml`	Completely normal in week six	You do not need to. uv edits it for you. Just know it records your packages

Check yourself

What is a package? Code somebody else wrote and shared, which you install once and then bring into your program with `import`.

What problem does an environment solve? Two projects needing different versions of the same package, which cannot both be installed in a single shared place.

What do the three uv commands do? `uv init` starts a project, `uv add` installs a package into the project's environment, `uv run` runs your code with those packages available.

What is `uv.lock` for? It records the exact version of every package your project uses, so anyone can rebuild an identical environment. That is what makes your project reproducible.

Why does `.venv` never go to GitHub? Because it can be rebuilt from `uv.lock` in seconds, and it contains thousands of files that belong to your machine, not to your project.

Your Week 6 deliverable

Upload to your Month 1 repository: `pyproject.toml`, `uv.lock` and `check.py`. Then open the `screenshots` folder on GitHub and upload `week6.png` into it. Do not upload `.venv`, `main.py` or the local `README.md`.

The screenshot shows the VS Code terminal after `uv --version` and then `uv run check.py`, with the pandas version printed.

Add to your README log:

```
- Week 6: set up the project with uv and added pandas. check.py prints the pandas version.
```

Submit on Google Classroom, on the Month 2 Week 6 task: your Month 1 repository link, `https://github.com/your-username/geoclean`, then click Turn in.

Week 7: Python, the six ideas

Your milestone this week: your first Python program, written entirely by you, that reads data and calculates something.

Preparation notes

Read this before the video. Fifteen minutes.

This is the most important week of the month, and one of the most important of the whole year. This is the week you learn to program.

Not completely. Nobody learns to program fully in a week. But by the end of this week you will understand the six ideas that every piece of code you will ever write is built from. Once you have these six, everything else is a variation on them.

The six ideas are: the **variable**, the **type**, the **list**, the **loop**, the **condition**, and the **function**. We will take them one at a time, slowly, with many small examples on data you recognise from Month 1.

Have VS Code open with your `geoclean` project, and a terminal inside it. Increase your editor's font size if it is small, with Ctrl and the plus key (Cmd and plus on Mac); you will be reading a lot of code this week.

One rule above all others this week: **type everything yourself.** Do not paste a single line. Your hands and eyes learn by typing. This is slower and it is the whole point.

What this week covers

The six ideas, each taught from nothing with examples, and then the crucial skill of reading an error message without panic.

Before the six ideas: how to try the examples

Every example in sections 1 to 6 is meant to be typed and run. Here is the routine, which you will use hundreds of times this year.

1. In VS Code, create a file called `learn.py` in your `geoclean` folder.
2. Type an example into it.

3. Save it (or let Auto Save do it).
4. In the terminal at the bottom, type `uv run learn.py` and press Enter.
5. Read what it prints. Change something, save, press the up arrow and Enter to run it again.

You can keep replacing the contents of `learn.py` with each new example. It is your scratch paper and is not handed in.

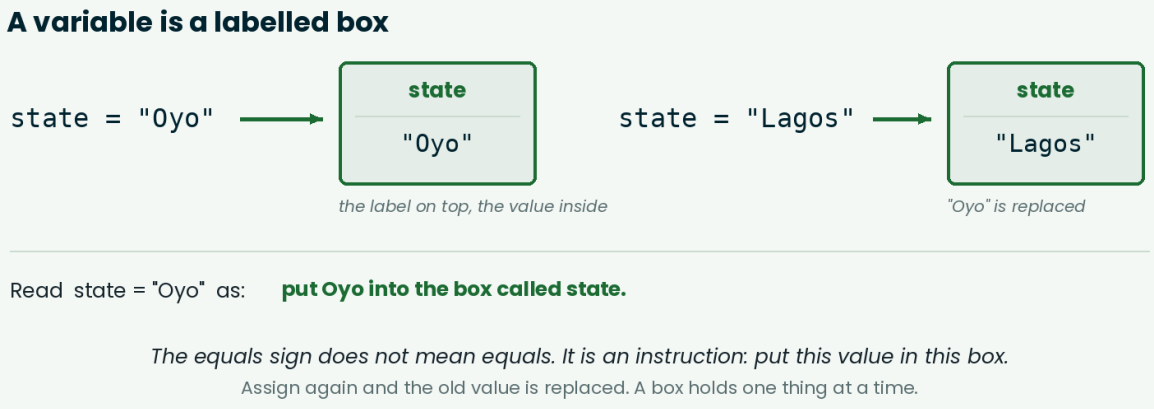
Three small things you will see in the examples:

`print(...)` shows whatever is inside the brackets on the screen. The brackets are required: `print "hello"` without brackets is an error in modern Python.

A `#` starts a comment. Everything after `#` on a line is a note for humans, and Python ignores it. The examples use comments to tell you what a line prints. You do not have to type the comments, though writing your own is a good habit.

Arithmetic uses `+` to add, `-` to subtract, `*` to multiply and `/` to divide. `28454 * 100` means 28454 times 100.

1. The variable: a labelled box



The equals sign does not mean equals. It is an instruction: put this value in this box.
Assign again and the old value is replaced. A box holds one thing at a time.

A **variable** is the most basic idea in programming. It is a labelled box that holds a value.

You create one by choosing a label, writing an equals sign, and giving it a value:

```
state = "Oyo"
```

This creates a box labelled `state`, and puts the text `"Oyo"` inside it. From now on, whenever you write `state`, Python looks in the box and uses what it finds.

```
state = "Oyo"
print(state)
```

This program prints `Oyo`, because that is what is in the `state` box. Notice `print(state)` has no quotes around `state`. Without quotes, Python treats the word as a box label and prints what is inside. With quotes, `print("state")`, it would print the word `state` itself.

Here is the first thing that surprises people. **The equals sign does not mean "equals" the way it does in mathematics.** It means "put this value into this box". It is an instruction, not a statement of fact. Read `state = "Oyo"` as "let state be Oyo" or "put Oyo into state".

Because it is an instruction, you can do it again, and the box takes a new value:

```
state = "Oyo"
print(state)
state = "Lagos"
print(state)
```

This prints `Oyo`, then `Lagos`. The second assignment replaced the first. A box holds one thing at a time; put something new in, and the old thing is gone. Python also runs your lines strictly from top to bottom, one at a time, which is why the first print shows Oyo.

Naming your boxes. You choose the labels, and good labels matter. Use lowercase letters, and put underscores between words. No spaces, and a name cannot start with a number. Make the name describe what is inside.

```
population = 5580894
buffer_distance = 500
ward_name = "Bodija"
```

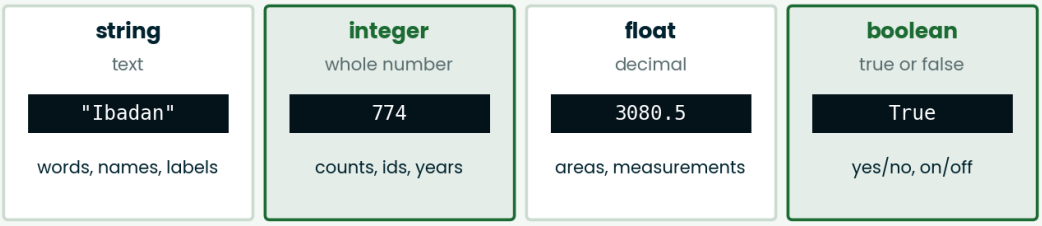
These are good names. `p`, `bd`, `w` are bad names, because in a month you will not remember what they meant. Code is read far more often than it is written, including by you, later. Name things so that future you understands them. Names are also case sensitive: `State` and `state` are two different boxes.

Try these in `learn.py`, run with `uv run learn.py`, change the values, run again, watch the output change. This is how you learn: type, run, observe.

2. The type: what kind of thing is in the box

Every value has a **type**, which is the kind of thing it is. The type decides what you can do with the value.

Four types, and what each one is for



The type decides what you can do. 5 + 5 is 10, but "5" + "5" is "55", because joining text is not adding.

Data read from a file always arrives as text. You convert it with int() or float() before doing maths.

There are four types you need this week.

A **string** is text. You write it inside quotes. `"Ibadan"`, `"Oyo State"`, `"borehole"`. Strings are for names, labels, any words. Single quotes and double quotes both work, as long as they match at both ends.

An **integer** is a whole number, with no decimal point. `774`, `36`, `5580894`. Integers are for counts, years, identifiers, anything you count in whole units. Write big numbers without commas: `5580894`, not `5,580,894`.

A **float** is a number with a decimal point. `3080.5`, `12.4`, `0.001`. Floats are for measurements, areas, distances, anything that can be fractional.

A **boolean** is either `True` or `False`, nothing else, always with a capital first letter. Booleans are for yes-or-no facts: is this ward coastal, is this value above the threshold, is the file empty.

```
ward_name = "Bodija"      # a string
ward_count = 774          # an integer
area_km2 = 3080.5         # a float
is_capital = False        # a boolean
```

You can always ask Python what type something is with `type()`: `print(type(area_km2))` prints `<class 'float'>`.

Why does the type matter? Because it decides what operations make sense. Here is the classic surprise. Type this and run it:

```
print(5 + 5)
print("5" + "5")
```

The first prints `10`, as you would expect. The second prints `55`. Why? Because `"5"` is a string, text, not a number. And "adding" two strings does not add them, it joins them end to end. `"5" + "5"` is `"55"` the same way `"Ibadan" + " North"` is `"Ibadan North"`. Joining text is a completely different operation from adding numbers, and the type is how Python tells them apart.

This matters enormously for real data, and here is why. **When you read data from a file, everything arrives as text, as strings.** The number `12.4` in your CSV comes into Python as the string `"12.4"`, not the number `12.4`. If you try to do maths on it while it is still text, you get nonsense or an error. You must first convert it to a number:

```
area_text = "12.4"                # this is a string, from a file
area_number = float(area_text)    # now it is a float, 12.4
print(area_number * 100)          # now maths works: 1240.0
```

`float()` converts text into a float. `int()` converts text into an integer. `str()` goes the other way and turns a number into text. You will use these constantly from Week 8 onward, every time you read a number from a file. Remember Month 1, where a number stored as text sorts wrongly in QGIS? This is the same fact, met again in code: text and numbers are different types, and you must convert between them deliberately.

3. The list: many things in order

A **list** holds many values in a single box, kept in order.

A list holds many things in order, counted from zero

```
wards = ["Agodi", "Bodija", "Sabo", "Mokola"]
```



The first item is at position zero, not one. It trips up everybody once, then feels normal.
wards[3] is Mokola. wards[4] is an IndexError, because there is no position four.

You make a list with square brackets, and commas between the items:

```
wards = ["Agodi", "Bodija", "Sabo", "Mokola"]
```

Now the box `wards` holds four values, in that order. You can ask how many there are with `len`, short for length:

```
print(len(wards))    # prints 4
```

And you can pull out a single item by its position, using square brackets with a number:

```
print(wards[0])    # prints Agodi
print(wards[1])    # prints Bodija
```

Here is the thing that catches everybody exactly once. Positions are counted from zero, not one. The first item is at position zero. The second is at position one. So `wards[0]` is Agodi, the first ward. `wards[3]` is Mokola, the fourth and last. The position number is called the **index**.

This feels wrong for about a week, and then it feels completely normal. Every programmer alive tripped on it once. Now you have been warned, so when your first item comes out as position zero, you will nod rather than panic.

Because there are four items, at positions zero to three, asking for `wards[4]` is an error: there is no position four. You will see an error called `IndexError` when this happens, and now you will know exactly what it means.

Lists can grow. You add an item to the end with `append`:

```
wards = ["Agodi", "Bodija", "Sabo"]
wards.append("Mokola")
print(wards)    # now shows all four
```

The dot in `wards.append` means “the append action that belongs to this list”. You will see this dot pattern often: `something.action()`.

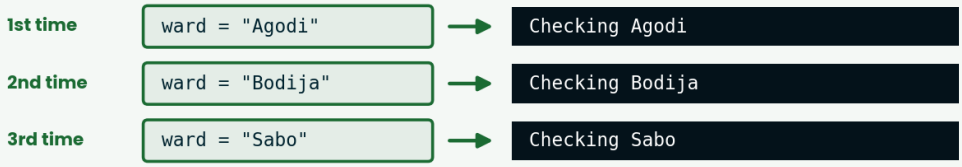
Lists are everywhere in real work, because data comes in many rows: many wards, many boreholes, many measurements. A list is how you hold them all together so you can process them, which is what the loop does next.

4. The loop: do something to each item

A **loop** does the same thing to every item in a list, one after another.

A loop does the same thing to each item in turn

```
for ward in wards:
    print("Checking", ward)
```



Each time round, the box called ward holds the next item. This is exactly what a batch process does.

This is the `for` loop, and it reads almost like English:

```
wards = ["Agodi", "Bodija", "Sabo"]

for ward in wards:
    print("Checking", ward)
```

Read that out loud: "for each ward in wards, print Checking, then the ward". And that is exactly what it does. It runs the indented line once for each item in the list. The first time round, the box `ward` holds `"Agodi"`. The second time, it holds `"Bodija"`. The third, `"Sabo"`. So it prints:

```
Checking Agodi
Checking Bodija
Checking Sabo
```

Two details make the loop work. The `for` line **ends with a colon**, `:`. And the line underneath is **indented**, pushed in by four spaces. Forget either and Python stops with an error.

The box `ward` is a fresh variable that the loop fills with each item in turn. You choose its name; `ward` is sensible here because each item is a ward. When you print two things separated by a comma, as in `print("Checking", ward)`, Python puts a space between them.

Here is the connection to Month 1. Every batch process you have ever run in a GIS toolbox is a loop. When you buffered every feature in a layer, the software looped over the features and buffered each one. You have been using loops for years without writing one. Now you write them yourself.

A loop can do real work, not just print:

```
areas = [28454, 3345, 20131]

for area in areas:
    hectares = area * 100
    print(area, "square km is", hectares, "hectares")
```

Each time round, it takes the next area, multiplies it, and prints the result.

Indentation matters, and it is not decoration. That indentation is how Python knows which lines are inside the loop and which are outside. Look at the difference:

```
for ward in wards:
    print(ward)       # indented: inside the loop, runs each time
print("done")         # not indented: outside, runs once at the end
```

The first print runs for every ward. The second runs once, after the loop finishes, because it is not indented. If you get the indentation wrong, Python either does the wrong thing or stops with an `IndentationError`. VS Code indents for you automatically when you press Enter after a line ending in a colon; let it help you. To come back out of the loop, press Backspace at the start of the new line.

5. The condition: choose a path

A **condition** lets your program choose between different actions depending on whether something is true.

This is the `if` statement:

```
area = 28454

if area > 20000:
    print("large ward")
else:
    print("small ward")
```

Read it: “if area is greater than 20000, print large ward, otherwise print small ward”. Because `area` is 28454, which is greater than 20000, this prints `large ward`.

The `if` line tests something. If the test is true, the indented lines under `if` run. If it is false, the indented lines under `else` run instead. Both `if` and `else` lines end with a colon, like the loop. The `else` part is optional; sometimes you only want to do something when the test is true, and nothing otherwise.

The tests use **comparison operators**, which mostly look like maths:

```
area > 1000      # greater than
area < 1000      # less than
area >= 1000     # greater than or equal to
area <= 1000     # less than or equal to
area == 1000     # equal to
area != 1000     # not equal to
```

One of these needs care: the double equals, `==`. A single equals sign puts a value into a box; that is assignment. A double equals asks "are these two things equal?"; that is a comparison. `state = "Oyo"` puts Oyo into state. `state == "Oyo"` asks whether state is Oyo, and gives back `True` or `False`. Mixing these up is a common mistake, and now you know the difference: **one equals to set, two equals to compare.**

When there are more than two paths, add `elif`, short for "else if", between them:

```
if area > 20000:
    print("large")
elif area > 5000:
    print("medium")
else:
    print("small")
```

Python checks each test from the top and runs the first one that is true.

Conditions are almost always used inside loops, to treat different items differently:

```
areas = [28454, 3345, 20131]

for area in areas:
    if area > 20000:
        print(area, "is large")
    else:
        print(area, "is small")
```

This checks each area in turn and prints whether it is large or small. Notice the indentation goes two levels deep now: the `if` is inside the loop, so it is pushed in four spaces, and the `print` is inside the `if`, so it is pushed in eight. Each level of "inside" is one more step of indentation.

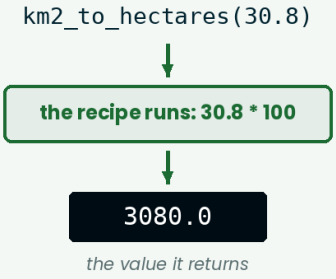
6. The function: a named, reusable recipe

A **function** is a named piece of code that takes an input and gives back a result. It is a recipe you write once and use many times. You have already used functions other people wrote: `print`, `len`, `float` and `type` are all functions.

A function is a named recipe: input goes in, a value comes out

```
def km2_to_hectares(area):
    return area * 100
```

1 the name you choose
2 the input it takes
3 what it hands back



Write the recipe once. Use it as many times as you like, with different inputs.

Here is why you want to write your own. Suppose you convert square kilometres to hectares in three places:

```
oyo_hectares = 28454 * 100
lagos_hectares = 3345 * 100
kano_hectares = 20131 * 100
```

The calculation, times 100, is repeated three times. If it were wrong, you would have to fix it in three places. In a real program it might be thirty places, and you would miss one. Repetition is where bugs hide.

A function fixes this. You define the recipe once:

```
def km2_to_hectares(area):
    return area * 100
```

Let us read that line by line. `def` tells Python you are defining a function. `km2_to_hectares` is the name you chose, following the same naming rules as variables. `(area)` says the function takes one input, which inside the function will be called `area`. The colon ends the definition line. The indented line is the body of the recipe: what it does. `return` sends a value back out.

Defining a function does not run it. Nothing happens until you use it, which is called **calling** the function:

```
print(km2_to_hectares(28454))    # prints 2845400
print(km2_to_hectares(3345))     # prints 334500
```

Each time you call it, you give it a number, the recipe runs with that number as `area`, and it returns the result. The calculation lives in one place now. Fix it once, and every call gets the fix. Always write the `def` above the lines that call it, because Python reads top to bottom and must meet the recipe before it can use it.

The word `return` is important, and it is the source of the most common beginner confusion, which you will meet properly in Week 8. `return` hands a value back out of the function so the rest of your code can use it. A function that calculates something almost always needs `return`, because otherwise the answer has nowhere to go.

Functions can take more than one input:

```
def describe(name, count):
    return name + " has " + str(count) + " wards"

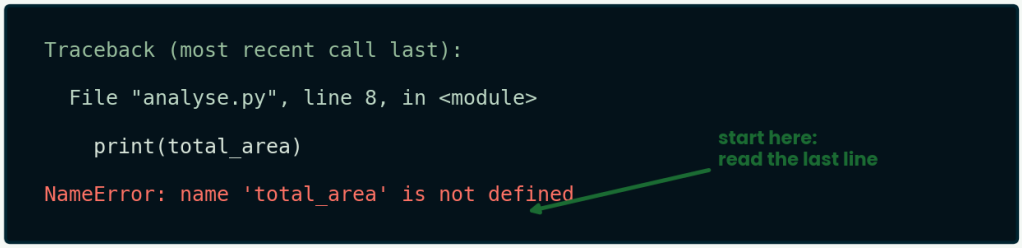
print(describe("Ibadan North", 12))
```

This takes two inputs and joins them into a sentence, `Ibadan North has 12 wards`. Note `str(count)`: because `count` is a number and we are joining it to text, we convert it to a string first, or Python will complain about mixing types, exactly as you saw in section 2. The values you pass in when calling, here `"Ibadan North"` and `12`, are called **arguments**.

Functions are how real programs are built. You break a big task into small named recipes, each doing one clear thing, and then combine them. Week 9 is entirely about this. For now, just get comfortable writing a simple function and calling it.

7. Reading an error without panic

How to read an error message



Python puts the real problem on the last line, in plain English. Just above, it tells you the line number.

An error is information, not failure. Read the bottom line, go to that line, fix it, run again.

This week you will see many errors. This is not a sign that you are bad at programming. It is what programming is. Professional developers see errors every single day, all day. The difference between a beginner and a professional is not that the professional avoids errors. It is that the professional reads them calmly instead of panicking.

An error message is the computer telling you exactly what went wrong, and usually where. Here is how to read one.

When your code fails, Python prints something like this:

```
Traceback (most recent call last):
  File "analyse.py", line 8, in <module>
    print(total_area)
NameError: name 'total_area' is not defined
```

It looks intimidating. It is not. Here is the method.

Read the last line first. Python puts the actual problem on the very last line, in reasonably plain English. Here it says `NameError: name 'total_area' is not defined`. That means: you used a name, `total_area`, that Python does not know about. Probably a typo, or a variable you never created.

Find the line number. Just above, it says `line 8`. That is the line in your file where it went wrong. VS Code shows line numbers down the left side of the editor. Go to line 8.

Fix it and run again. On line 8 you would find `print(total_area)`, and you would realise you meant `print(total)`, or that you never made `total_area`. Fix it, save, run again.

That is the whole method: read the bottom line, find the line number, go there, fix it, run again. It works for almost every error you will ever meet. Sometimes the real mistake is on the line just before the one named, especially a missing bracket or quote, so look one line up as well.

Four errors you will see this week, so you recognise them:

`NameError` means you used a name Python does not know. A typo, or a variable you forgot to create.

`TypeError` means you tried to do something to the wrong type. Usually joining text and a number, like `"Count: " + 5`, which should be `"Count: " + str(5)`.

`IndentationError` means your spaces are wrong. A line is indented when it should not be, or not indented when it should be.

`SyntaxError` means Python cannot read the line at all: a missing colon at the end of a `for`, `if` or `def` line, a missing closing bracket, or quotes that do not match.

None of these mean you have failed. Each one is a small, specific, fixable problem, and the message tells you what it is. Read it, fix it, move on.

Guided practical

Outcome: a Python program you wrote yourself, that reads a small data file, processes each row, and calculates a result.

You will build this up in small steps, running after each one. Each step shows the **whole file**: replace what is in `analyse.py` with the new version each time. Type everything. Do not paste.

Step 1: Make a small data file

In VS Code, in your `geoclean` project, create a file called `wards.csv` and type in these lines. A CSV is just text, with values separated by commas:

```
ward,area_km2
Agodi,12.4
Bodija,8.1
Sabo,5.7
Mokola,9.3
```

Save it. The first line is the **header**, naming the columns. The rest are data rows. Do not add spaces after the commas, and do not leave an empty line at the end of the file. After `9.3`, stop typing.

Step 2: Read the file and count the lines

Create a file called `analyse.py` and type:

```
with open("wards.csv") as f:
    lines = f.readlines()

print("Number of lines:", len(lines))
```

`open("wards.csv")` opens the file. The `with ... as f:` part gives the open file the short name `f` and makes sure the file is closed properly when you are done, which is good practice; just always write it this way. `f.readlines()` reads all the lines into a list. Then you print how many.

Run it:

```
uv run analyse.py
```

You should see `Number of lines: 5`: four wards plus the header.

Step 3: Print each line

Add a loop to look at each line:

```
with open("wards.csv") as f:
    lines = f.readlines()

for line in lines:
    print(line.strip())
```

`line.strip()` removes the invisible "new line" character at the end of each line, so the output is tidy. Run it, and you will see each line printed.

Step 4: Skip the header and split each row

The first line is the header, not data, so skip it. And each data line needs splitting at the comma into its parts:

```
with open("wards.csv") as f:
    lines = f.readlines()

for line in lines[1:]:
    parts = line.strip().split(",")
    ward = parts[0]
    area = parts[1]
    print(ward, "has area", area)
```

Two new things. `lines[1:]` means "from position one to the end", which skips the header at position zero. This is called a **slice**. And `split(",")` breaks a line into a list of pieces at each comma, so `"Agodi,12.4"` becomes `["Agodi", "12.4"]`. Then `parts[0]` is the ward name and `parts[1]` is the area. Run it and check each row prints correctly.

Step 5: Convert the area to a number and calculate

The area is still text at this point. Convert it to a number so you can do maths:

```
with open("wards.csv") as f:
    lines = f.readlines()

for line in lines[1:]:
    parts = line.strip().split(",")
    ward = parts[0]
    area_km2 = float(parts[1])
    area_hectares = area_km2 * 100
    print(ward, "is", area_hectares, "hectares")
```

`float(parts[1])` converts the text `"12.4"` into the number `12.4`, so the multiplication works. Run it and you will see each ward's area in hectares. If you see a number like

`930.0000000000001` for Mokola, that is normal: computers store decimals with tiny rounding differences. `round(area_hectares, 1)` tidies it if it bothers you.

Step 6: Add a decision

Now use a condition to classify each ward:

```
with open("wards.csv") as f:
    lines = f.readlines()

for line in lines[1:]:
    parts = line.strip().split(",")
    ward = parts[0]
    area_km2 = float(parts[1])
    if area_km2 > 9:
        size = "large"
    else:
        size = "small"
    print(ward, "is", size)
```

You should see:

```
Agodi is large
Bodija is small
Sabo is small
Mokola is large
```

You now have a program that reads a file, loops over the data, converts types, and makes a decision for each row. That is real programming, using five of the six ideas together. Read it and make sure you understand every line. This version is your Week 7 deliverable.

Step 7: Break it on purpose, and read the error

Change `float(parts[1])` to `int(parts[1])` and run it. You will get a `ValueError`, because `12.4` is not a whole number and `int` cannot convert it. Read the error. Notice it names the line and explains the problem. Then change it back to `float` and run it once more to confirm it works again.

Deliberately breaking something and reading the error is one of the best ways to learn. You are training yourself to read errors calmly, which is the skill that will carry you through the whole year.

Step 8: Hand in

Make sure `analyse.py` is back to the working version from Step 6. Take your screenshot and upload, as described in the deliverable box below. You have written your first real program.

You are done when

- You wrote `analyse.py` yourself, typing every line
- It reads the file, skips the header, splits each row, converts the area, and classifies each ward
- You understand what every single line does
- You triggered at least one error on purpose and read it calmly
- Your Week 7 deliverable is on GitHub and turned in

Independent challenge

Only if the practical felt comfortable. Not graded.

Add a running total. Before the loop, create a variable `total = 0`. Inside the loop, add each ward's area to it with `total = total + area_km2`. After the loop, print the total. Be careful where you put the final `print`: if it is indented inside the loop, it prints every time round; it should be outside, at the end, so it prints once. This teaches you the difference indentation makes. The answer should be `35.5`.

Then, harder. Turn the size decision into a function. Write `def classify(area):` that returns `"large"` or `"small"`, and call it inside the loop with `size = classify(area_km2)`. This is the same logic, reorganised into a reusable recipe, which is exactly what Week 9 is about.

When it breaks

What you see	What it means	What to do
`FileNotFoundError`	Python cannot find `wards.csv`	Make sure the file is in the same folder, and you are running from there. Check with `ls`

What you see	What it means	What to do
`IndentationError`	Your spaces are inconsistent	Let VS Code indent for you. Four spaces per level. Never mix tabs and spaces
`SyntaxError: expected ':'`	A `for`, `if`, `else` or `def` line has no colon at the end	Add the colon
`SyntaxError` pointing at a quote or bracket	A quote or bracket was opened and never closed	Count them on that line and the one above
`ValueError: could not convert`	You tried to make a number from text that is not one	Check the row; a stray letter or space breaks conversion. Or you used `int` on a decimal
`IndexError: list index out of range`	You asked for a position that does not exist	Usually an empty line at the end of `wards.csv`. Delete it. Or a zero-based counting mistake
`NameError`	A name is misspelt, or used before it was created	Check spelling and capitals; make sure the line creating it comes first
The final total is wrong or repeats	Your print is inside the loop	Move it outside, so it is not indented under `for`
Nothing prints	The loop may be reading zero rows, or the file is not saved	Check the file has content, is saved, and the name is spelled right

Check yourself

What are the six ideas? Variable, type, list, loop, condition, function.

Why does the first item in a list have position zero? Python counts positions from zero. `list[0]` is the first item. It catches everyone once, then feels normal.

Why must you use `float()` on a number read from a file? Because data read from a file arrives as text, and you cannot do maths on text. `float()` converts it to a number.

What is the difference between = and ==? A single equals puts a value into a box. A double equals asks whether two things are equal, and gives back `True` or `False`.

How do you read an error? Read the last line first for the problem, find the line number just above it, go to that line, fix it, run again.

Your Week 7 deliverable

> **Upload to your Month 1 repository:** `wards.csv` and `analyse.py`, the working version from Step 6. Then upload `week7.png` into the `screenshots` folder. You do not need to upload `learn.py`.
>
> **The screenshot shows** the VS Code terminal after `uv run analyse.py`, with all four wards classified as large or small.
>
> **Add to your README log:** `- Week 7: analyse.py reads wards.csv, converts each area and classifies each ward.`
>
> **Submit on Google Classroom,** on the Month 2 Week 7 task: your Month 1 repository link, `https://github.com/your-username/geoclean`, then click Turn in.

Week 8: Python, files and data

Your milestone this week: a data inspection script that reads your own Month 1 dataset and reports something useful about it.

Preparation notes

Read this before the video. Fifteen minutes.

Last week you learned the six ideas on a tiny made-up file. This week you point them at real data, the data you gathered and cleaned in Month 1, and you learn to handle the formats real data actually comes in.

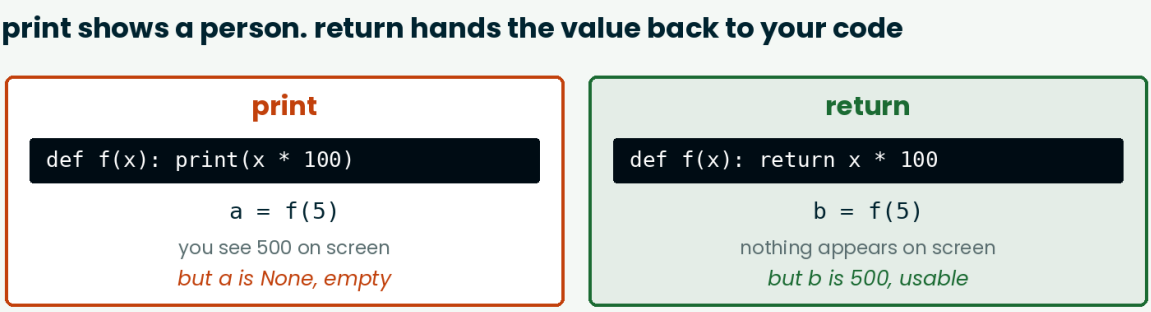
You also meet the single most common beginner mistake in all of Python, the difference between `print` and `return`. Understanding it this week will save you weeks of confusion later, so we lead with it.

Have your `geoclean` project open, and find a CSV from your Month 1 work, or be ready to export one from QGIS. Step 1 of the practical shows how. This is the week Month 1 and Month 2 connect.

What this week covers

The print-versus-return distinction, reading CSV and JSON properly with the right tools, dictionaries, debugging a script that misbehaves, and handling the messy and missing values that real data always contains.

1. print shows a person, return hands the value back



The most common beginner mistake. print is for a person. return is for your code.

When a variable is mysteriously None, look for a missing return first.

This is the mistake almost every beginner makes. Understanding it now, clearly, puts you ahead of most people who learn programming alone.

`print` puts something on the screen for a person to read. That is all it does. Once printed, the value is gone; your code cannot use it.

`return` sends a value back out of a function, so the rest of your code can use it. Nothing appears on screen; the value is handed to whatever called the function.

Look at the two side by side. Type this into `learn.py` and run it:

```
def with_print(x):
    print(x * 100)

def with_return(x):
    return x * 100

a = with_print(5)
b = with_return(5)

print("a is", a)
print("b is", b)
```

When you run it, you see:

```
500
a is None
b is 500
```

Let us understand exactly what happened.

`with_print(5)` calculated `5 * 100`, which is 500, and printed it. That is the `500` at the top. But it did not return anything. So when you wrote `a = with_print(5)`, there was no value to put in `a`. Python put `None` in it, which means "nothing". That is why `a is None`.

`with_return(5)` calculated 500 and returned it, but did not print it. So nothing appeared on screen from that call. But the returned value, 500, was put into `b`. That is why `b is 500`.

The rule: if you want to see something, use `print`. If you want your code to use a value later, use `return`. A function that calculates a result almost always needs `return`, because the whole point is to hand that result back.

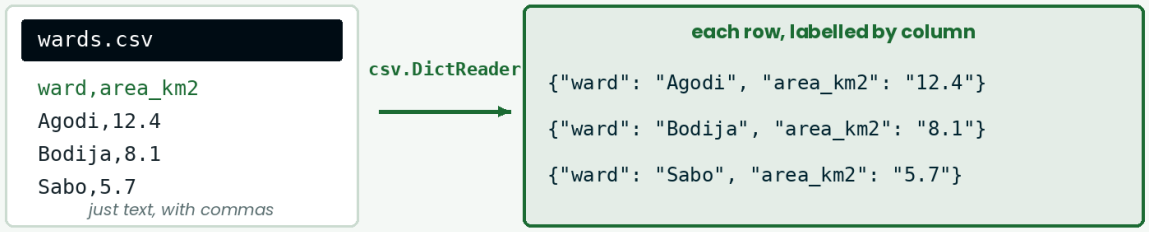
And the practical consequence, which will save you real time: when a variable is mysteriously empty, or `None`, or a calculation seems to vanish, the cause is very often a function that printed instead of returning. When something is unexpectedly empty, check for a missing `return` first. You will be right most of the time.

2. Reading a CSV properly

Last week you split CSV lines by hand, using `split(",")`. That works for simple files, but it breaks the moment a value itself contains a comma. Imagine a place written `"Ibadan, Oyo"`. Splitting that line at every comma would wrongly break the place name in two.

Python has a proper tool for this, the `csv` module, and it handles the awkward cases for you. A **module** is a file of ready-made code; `csv` comes with Python, so you do not need to `uv add` it.

Reading a CSV: text on disk becomes rows you can work with



You ask for row["ward"] by name, instead of splitting by hand and counting commas.
Notice "12.4" is still in quotes: every value arrives as text until you convert it.

Here is the better way to read a CSV:

```
import csv

with open("wards.csv", encoding="utf-8-sig") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["ward"], row["area_km2"])
```

`import csv` brings in the CSV tool. `csv.DictReader(f)` reads the file and hands you each row as a labelled collection, using the header line to name the columns. So instead of `parts[0]` and `parts[1]`, counting positions, you write `row["ward"]` and `row["area_km2"]`, asking for columns by name. The name inside the brackets must match the header exactly, including capital letters.

What is `encoding="utf-8-sig"`? An encoding is the rule a file uses to store letters as numbers. CSVs saved from Excel, and some exported from QGIS, begin with an invisible marker. Without this setting, that marker sticks to the first column name, so `row["ward"]` fails with a `KeyError` even though the column is plainly there. Adding `encoding="utf-8-sig"` quietly removes the marker, and works just as well on files that do not have one. From now on, open every CSV this way.

This is clearer to read, it does not break on commas inside values, and it does not care what order the columns are in. From now on, this is how you read CSVs. The labelled collection that `DictReader` gives you for each row is called a **dictionary**, which is the next idea.

3. The dictionary: values by label

A dictionary holds values by label, not by position

```
ward = {"name": "Bodija", "area": 8.1, "lga": "Ibadan North"}
```



A list finds things by number: wards[0]. A dictionary finds them by name: ward["name"].

Each row from DictReader was a dictionary. That is why you wrote row of area_km2 in quotes.

You met lists in Week 7. A list holds values in order, and you get them by position: `wards[0]`. A **dictionary** holds values by label, and you get them by name.

You make a dictionary with curly braces, and pairs of label and value, with a colon between each label and its value:

```
ward = {"name": "Bodija", "area": 8.1, "lga": "Ibadan North"}
```

Each entry has a label, called a **key**, and a **value**. `"name"` is a key, `"Bodija"` is its value. You get a value by giving its key in square brackets:

```
print(ward["name"])    # prints Bodija
print(ward["area"])    # prints 8.1
```

A list is right when order matters and items are all the same kind of thing, like a list of ward names. A dictionary is right when each value means something different and you want to look them up by name, like the properties of one ward.

This connects directly to the last section. Each `row` that `DictReader` gave you was a dictionary. That is exactly why you wrote `row["ward"]`, with the column name in quotes: you were looking up a value in a dictionary by its key. Now you know what that row really was.

You will meet dictionaries constantly, because they are how structured data is represented. The attribute table of a single feature in QGIS is essentially a dictionary: field names are keys, and that feature's entries are values. Every feature in a GeoJSON file is stored the same way. When you reach spatial data in code next month, this idea returns.

4. Reading JSON

A great deal of geographic data, including GeoJSON, comes in a format called **JSON**. It is worth a short introduction now, because you will meet it properly next month.

JSON looks like nested Python lists and dictionaries, which is convenient, because that is exactly what Python turns it into:

```
import json

with open("data.json", encoding="utf-8") as f:
    data = json.load(f)

print(data["name"])
print(data["features"][0])
```

This example is for reading only; you do not need a `data.json` file this week. `json.load(f)` reads a JSON file and hands you back ordinary Python dictionaries and lists. A JSON object, written in curly braces, becomes a dictionary you access by key. A JSON array, written in square brackets, becomes a list you loop over or index by position. So `data["features"][0]` means: in the dictionary `data`, get the value under the key `"features"`, which is a list, and from that list get the first item.

You do not need to master JSON this week. You need to know that `json.load` opens it, and that what you get back is the lists and dictionaries you already understand. That is enough for now.

5. Debugging: see what the code is actually doing

Debugging: stop guessing, and see what the code is actually doing

```
for row in reader:
    print("about to process:", row)                    <-- a print you added
    area = float(row['area_km2'])
    print("area is:", area)                            <-- another one
    # ... now you can see exactly where it breaks
```

When a script does the wrong thing, add print statements until you can see where reality stops matching what you expected. Then remove them. It is not a special skill, it is a habit.

Sooner or later your script will do something you did not expect. It will produce the wrong number, or nothing, or crash. This is normal. The skill of finding out why is called **debugging**, and it is not mysterious. It is a habit: when something is wrong, stop guessing and find out what the code is actually doing, rather than what you assumed it was doing.

The simplest debugging tool is one you already know: `print`. Put a `print` wherever you are unsure, to see the real value at that point:

```
import csv

with open("wards.csv", encoding="utf-8-sig") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print("about to process:", row)
        area = float(row["area_km2"])
        print("area is:", area)
```

Those two prints show you exactly what each row contains, and whether the conversion to a number succeeded. If the script crashes on a particular row, the prints before the crash show you which row and what was in it. Very often, the moment you can see the real values, the problem is obvious.

When a script misbehaves, add prints until you can see the point where reality stops matching your expectation. That point is where your bug is. Then remove the prints once it works. This one habit will solve most of the problems you hit this year.

VS Code also has a proper debugger, which lets you pause your code and inspect everything. It is worth learning later. For now, `print` is enough, and honestly it is what most experienced developers reach for first anyway.

6. Handling messy and missing data

Real data is never clean. Month 1 taught you this with your eyes; now you handle it in code. The most common problem is a missing value: a cell that is empty.

If you try to convert an empty cell to a number, it fails:

```
float("")    # ValueError: could not convert string to float: ''
```

So you must check for it and decide what to do. Usually you skip that row:

```
import csv

with open("wards.csv", encoding="utf-8-sig") as f:
    reader = csv.DictReader(f)
    for row in reader:
        raw = row["area_km2"].strip()
        if raw == "":
            continue
        area = float(raw)
        print(area)
```

Two things are new. `.strip()` removes any spaces around the value, so a cell containing only a space also counts as empty. And `continue` means "skip the rest of this turn of the loop and go straight to the next item". So when the value is empty, `continue` jumps to the next row without trying to convert the empty string. Empty cells no longer crash your script.

This is the null-versus-zero idea from Month 1, met again in code. An empty cell is not zero. It is missing. Skipping it is honest; treating it as zero would quietly corrupt your results, exactly as it would have in QGIS.

Two other kinds of mess you are likely to meet in Nigerian datasets: numbers written with thousands separators, like `5,580,894`, and placeholder text such as `NA`, `N/A` or `-` in place of a value. Both make `float` fail. The first is fixed with `raw.replace(",", "")` before converting. The second you treat like an empty cell and skip. The When it breaks table shows both.

Guided practical

Outcome: a script that reads your own Month 1 data, cleans it, and reports a real summary, using a function that returns its result.

This week you work on your own data, connecting Month 2 back to Month 1.

Step 1: Get your own data into the project

Find a CSV from your Month 1 work. If you have one on your computer, copy it into your project folder using File Explorer or Finder.

If it is only on GitHub, in your Month 1 repository, download it: on your repository page click the CSV file, then click the **Download raw file** button (a small downward arrow near the top right of the file). Move the downloaded file from your Downloads folder into your project folder.

If your data is in QGIS as a layer, export it: in the **Layers** panel, right-click the layer, choose **Export**, then **Save Features As**. Set **Format** to **Comma Separated Value [CSV]**, click the **…** button beside **File name**, go to your `geoclean` folder, type a name, and click **Save**, then **OK**.

Give the file a short name with no spaces, such as `mydata.csv` or `oyo_boreholes.csv`. A name like `Oyo Boreholes Final (2).csv` works in Python but causes trouble in the terminal and on GitHub. Wherever this pack writes `mydata.csv`, use your file's name.

Your file needs at least **two columns of numbers**, such as area, population, depth, temperature or count. If yours has none, add one in QGIS with the field calculator, such as area with `$area`, before exporting.

If you genuinely have no suitable CSV, use the `wards.csv` from Week 7, expanded with a few more rows and a second number column, such as `population`, so there is something to summarise. Say so in your README.

Step 2: Read it with DictReader and count the rows

Create a file called `inspect_data.py` and type:

```
import csv

with open("mydata.csv", encoding="utf-8-sig") as f:
    reader = csv.DictReader(f)
    rows = list(reader)
```

```
print("Number of rows:", len(rows))
print("Columns:", list(rows[0].keys()))
```

> *Why inspect_data.py and not inspect.py? Python already has a built-in module called inspect. If you name your own file inspect.py, Python can pick up your file instead of its own, and other code breaks with baffling errors. The same goes for csv.py, json.py, pandas.py and any other package name. Never name a file after a module you might import.*

Change `"mydata.csv"` to your file's name. `list(reader)` reads all the rows into a list so you can use them more than once. `rows[0].keys()` shows the column names, taken from the header. Run it:

```
uv run inspect_data.py
```

Confirm the count and columns match your data. Copy the exact column names from this output; you need them in the next step.

Step 3: Inspect one number column

Pick a column of numbers from your data. Report its smallest and largest values:

```
import csv

with open("mydata.csv", encoding="utf-8-sig") as f:
    rows = list(csv.DictReader(f))

values = []
for row in rows:
    value = float(row["area_km2"])
    values.append(value)

print("Smallest:", min(values))
print("Largest:", max(values))
print("Count:", len(values))
```

Change `"area_km2"` to your column's exact name. `values = []` starts an empty list, and each turn of the loop appends one number to it. `min`, `max` and `len` are built into Python. This is a real data inspection, the kind you did by eye in Month 1, now done in code that you could run again on new data in seconds.

Step 4: Handle the empty cells

Real data has gaps, and Step 3 will crash if any cell in your column is empty. Guard against it:

```
import csv

with open("mydata.csv", encoding="utf-8-sig") as f:
    rows = list(csv.DictReader(f))

values = []
for row in rows:
    raw = row["area_km2"].strip()
    if raw == "":
        continue
    values.append(float(raw))

print("Smallest:", min(values))
print("Largest:", max(values))
print("Count:", len(values))
```

Now empty cells are skipped rather than crashing the script. Run it and confirm it works on your real data. If it still crashes with a `ValueError`, read the value named at the end of the error and check the When it breaks table.

Step 5: Turn it into a function that returns

Now wrap the inspection into a reusable function that hands back a result, applying the print-versus-return lesson:

```
import csv

def inspect_column(filename, column):
    with open(filename, encoding="utf-8-sig") as f:
        rows = list(csv.DictReader(f))
    values = []
    for row in rows:
        raw = row[column].strip()
        if raw == "":
            continue
        values.append(float(raw))
    return {
        "smallest": min(values),
        "largest": max(values),
        "count": len(values),
    }

result = inspect_column("mydata.csv", "area_km2")
print(result)
```

Notice the function uses `return`, not `print`. It hands back a dictionary of results, and the code outside decides to print it. This is the distinction from section 1, applied in practice. Notice also that the function takes the filename and column as inputs, so you could call it on any file and any column, without rewriting it. Inside the function, `column` is a variable holding the column name, so `row[column]` has no quotes around `column`.

You should see something like `{'smallest': 5.7, 'largest': 12.4, 'count': 4}`, with your own numbers.

Step 6: Use your function twice

Because it is a reusable function, inspecting a second column is almost no work. Replace the last two lines of the file with:

```
area_result = inspect_column("mydata.csv", "area_km2")
pop_result = inspect_column("mydata.csv", "population")

print("Area:", area_result)
print("Population:", pop_result)
```

Change the column names to two real number columns in your data, and the labels in the `print` lines to match. This is the payoff of writing a function: you wrote the logic once, and now you reuse it freely. This finished file is your Week 8 deliverable.

Step 7: Hand in

Save your script, take the screenshot, and upload as described in the deliverable box below. You have now inspected your own Month 1 data with code you wrote.

You are done when

- Your script reads your own Month 1 data with `DictReader`
- It reports the smallest, largest and count of a real column
- Empty cells are skipped rather than crashing it
- The inspection is a function that returns a result, and you called it on two columns
- You understand why the function uses `return` and not `print`
- Your Week 8 deliverable is on GitHub and turned in

Independent challenge

Only if the practical felt comfortable. Not graded.

Add the average, safely. Inside your function, calculate the mean with `sum(values) / len(values)` and add it to the returned dictionary. Then think about what happens if the column has no valid values at all: `len(values)` would be zero, and dividing by zero crashes. Guard against it: if there are no values, return a result that says so honestly rather than crashing.

Then, harder. Make your function cope with a column that might not be numbers at all. For each value, try to convert it; if the conversion fails, count it as non-numeric instead of crashing. You will need `try:` and `except ValueError:`, which you can look up in the Python documentation or ask your pod about. Return both the numeric summary and a count of how many values could not be converted. This is the beginning of real data cleaning, and it is exactly the kind of honest data handling Month 1 asked you to do by eye.

When it breaks

What you see	What it means	What to do
`KeyError: 'area_km2'`	That column name is not in your file	Print `rows[0].keys()` and use the exact name, including capitals and underscores
`KeyError` although the column is clearly there	An invisible marker is stuck to the first column name	Open the file with `encoding="utf-8-sig"`, as in section 2
`ValueError: could not convert string to float: ''`	A cell is empty	Add the empty-cell skip from Step 4
`ValueError: could not convert string to float: '5,580,894'`	The number has thousands separators	Convert with `float(raw.replace(",", ""))`
`ValueError` naming `'NA'`, `'N/A'` or `'-'`	Placeholder text instead of a value	Treat it as missing: `if raw in ("", "NA", "N/A", "-"): continue`
`ValueError` saying `min()` got an empty argument	Every value in the column was skipped	Check you picked a column that really holds numbers
`UnicodeDecodeError`	The file uses an older encoding	Try `encoding="latin-1"` in `open`, or re-export from QGIS with UTF-8 selected
Strange errors mentioning `inspect` or `csv`	You named your file after a Python module	Rename it, for example `inspect_data.py`, and delete any `__pycache__` folder
`ZeroDivisionError`	You averaged zero values	Check the list is not empty before dividing

What you see	What it means	What to do
The script prints nothing	The function returns, but you never printed the result	`print(result)` outside the function
`None` where a number should be	The function used `print` instead of `return`	Change `print` to `return` inside the function
`FileNotFoundError`	The filename is wrong or the file is elsewhere	Check the exact name with `ls`, and that the file is in your project folder

Check yourself

What is the difference between `print` and `return`? `print` shows a value to a person. `return` hands a value back to the code. A function that calculates a result needs `return`.

Why use `csv.DictReader` instead of splitting by hand? It handles values that contain commas, lets you access columns by name, and does not care about column order.

What is a dictionary, and how does it differ from a list? A dictionary holds values by label; you get them by name, `ward["name"]`. A list holds values in order; you get them by position, `wards[0]`.

What does `continue` do in a loop? It skips the rest of the current turn and moves straight to the next item. It is how you skip a bad or empty row.

How do you debug a misbehaving script? Add `print` statements to see the real values, until you find where reality stops matching your expectation. That is where the bug is.

Your Week 8 deliverable

Upload to your Month 1 repository: `inspect_data.py`, and your data CSV (for example `mydata.csv`) unless that exact file is already in the repository from Month 1. If you renamed or trimmed it, upload the new copy. Then upload `week8.png` into the `screenshots` folder. Check the Deliverables section first: remove any personal data, and use a smaller copy if the file is over 25 MB.

The screenshot shows the VS Code terminal after `uv run inspect_data.py`, with the summary dictionaries for two of your columns printed.

Add to your README log: one line naming your dataset and the two columns you summarised, for example `- Week 8: inspect_data.py summarises depth and yield in my Oyo boreholes data.`

Submit on Google Classroom, on the Month 2 Week 8 task: your Month 1 repository link, `https://github.com/your-username/geoclean`, then click Turn in.

Week 9: Consolidation and refactoring

Your milestone this week: your Week 8 script refactored into clean functions, producing the same result, plus a capstone script that summarises your own Month 1 data. This is also your Month 2 monthly task.

Preparation notes

Read this before the video. Ten minutes.

This week introduces almost no new ideas. That is deliberate and important. You have learned an enormous amount in four weeks, and this week is for making it solid before Month 3 builds directly on top of it.

The one new skill is **refactoring**: improving code that already works, without changing what it does. It is one of the most valuable habits a developer has, and it is a gentle, satisfying way to consolidate everything you have learned by improving your own earlier code.

This is a two-mentor week. One mentor walks through refactoring a script from start to finish. The other holds office hours, working through your own code with you. Come to the office hours with something that is not quite working, if you have it.

Have your Week 8 `inspect_data.py` open.

What this week covers

What clean code is and why it matters, a proper look at two ideas you have already half-used, the common errors gathered in one place, refactoring a real script step by step, and a capstone that combines the whole month on your Month 1 data.

1. What refactoring is, and why it matters

Your Week 8 script works. But “works” and “good” are not the same thing. Good code is code that another person, including you in six months, can read, understand and change without fear of breaking it.

Refactoring: same behaviour, better shape

BEFORE

```
with open('d.csv') as f:
  rows=list(csv.DictReader(f))
vals=[]
for r in rows:
  if r['a']=='': continue
  vals.append(float(r['a']))
print(min(vals),max(vals),
      sum(vals)/len(vals))
```

one long block, hard to reuse

AFTER

```
rows = read_rows('d.csv')
vals = clean_values(rows,'a')
result = summarise(vals)
print(result)

# each job is its own small
# function you can reuse and test
```

reads like a description of the program

You never change what the code does. You run it before and after and confirm the output is identical.

Clean code has fewer bugs and is far faster to change. Developers refactor constantly.

Refactoring means improving the shape of working code: giving variables clearer names, splitting one long block into small functions that each do a single job, removing repeated code, and adding a short comment where the reason for something is not obvious.

The iron rule of refactoring is this: **you never change what the code does.** You run it before you start, note the output, refactor, run it again, and confirm the output is identical. You improved the shape while keeping the behaviour exactly the same. If the output changed, you did not refactor, you broke something.

Why bother, if the code already works? Because clean code has far fewer bugs, and because you change code constantly. Requirements shift, data changes, you think of a better way. Clean code is fast and safe to change. Messy code is slow and dangerous to change. Developers refactor all the time, not out of tidiness for its own sake, but because it makes every future change easier.

2. Two ideas you have already been using

Before refactoring, let us properly name two things you met in passing, so you use them with understanding.

Dictionaries, revisited. In Week 8 you learned that each row from `DictReader` is a dictionary, a collection of values looked up by name. It is worth seeing them made and changed directly:

```
ward = {"name": "Bodija", "area": 8.1, "lga": "Ibadan North"}
print(ward["name"])           # look up by key
```

```
ward["population"] = 45000     # add a new key and value
print(ward)                    # now has four entries
```

You create a dictionary with curly braces and key-value pairs, look things up by key in square brackets, and add new entries by assigning to a new key. You can also start with an empty one, `report = {}`, and fill it as you go, which the capstone does. Your `inspect_column` function in Week 8 returned a dictionary: `{"smallest": ..., "largest": ..., "count": ...}`. Now you know exactly what that was and could build one yourself.

To go through every key and value in a dictionary, use `.items()` in a loop:

```
for key, value in ward.items():
    print(key, "->", value)
```

Each turn of the loop fills two boxes at once, `key` and `value`. The capstone uses exactly this.

Comprehensions. In tutorials online you will often see a compact way to build a list, like `values = [float(row["area"]) for row in rows]`. That is called a **comprehension**. It is a shorter way to write a common loop. These two pieces of code do exactly the same thing:

```
# the long way
values = []
for row in rows:
    values.append(float(row["area"]))

# the short way, a comprehension
values = [float(row["area"]) for row in rows]
```

Both build a list of numbers from the rows. The comprehension is more compact once you are used to reading it. The long form is never wrong, so use whichever is clearer to you. Recognise comprehensions when you see them, because they are common, but do not feel obliged to write them.

3. The common errors, gathered in one place

Now that you have hit these errors yourself, it helps to see them together, so you recognise each instantly.

Confusing print and return. A function that prints but does not return hands back `None`. If a variable is unexpectedly `None`, look for a missing `return`. This is the big one.

Indentation gone wrong. Lines inside a loop or function must be indented consistently. A `print` in the wrong place runs too often or too rarely. VS Code helps, but check your indentation when behaviour is strange.

Zero-based positions. The first item is at position zero. The last valid position is one less than the length. Asking for `list[len(list)]` is always an error.

Type mistakes. Data from a file is text. Convert it with `float` or `int` before doing maths. Joining text and a number without converting gives a `TypeError`.

Reusing a name by accident. If you use the same variable name for two different things, the second quietly overwrites the first. Give different things different names. The same applies to file names: never name a file after a module you import.

Running an old version. An unsaved file runs as it was at the last save. If a change seems to have no effect, check the tab for the unsaved dot.

None of these are signs of failure. They are the normal texture of programming, and now you can name and fix each one.

Guided practical

Outcome: your Week 8 script refactored into clean functions with identical output, then a capstone that summarises your whole Month 1 dataset.

Step 1: Run your Week 8 script and record its output

Before changing anything, run your Week 8 inspection script:

```
uv run inspect_data.py
```

Copy its output somewhere safe: select it in the terminal, copy it, and paste it into a new file called `notes.txt`. This is your reference. After refactoring, the core numbers must match this exactly. Do not change `inspect_data.py` itself this week; it stays as your Week 8 deliverable.

Step 2: Identify the separate jobs

Read your script and list the distinct jobs it does. A typical Week 8 script does three:

1. Read the file into rows
2. Extract and clean the values from a column
3. Summarise those values

Each of these should become its own small function. A function that does one clear job is easy to name, easy to understand, and easy to fix.

Step 3: Write one function per job

Create a fresh file, `summary.py`, and build the three functions:

```
import csv

def read_rows(filename):
    with open(filename, encoding="utf-8-sig") as f:
        return list(csv.DictReader(f))

def clean_values(rows, column):
    values = []
    for row in rows:
        raw = row[column].strip()
        if raw == "":
            continue
        values.append(float(raw))
    return values

def summarise(values):
    if len(values) == 0:
        return {"count": 0, "smallest": None, "largest": None, "average": None}
    return {
        "count": len(values),
        "smallest": min(values),
        "largest": max(values),
        "average": sum(values) / len(values),
    }
```

Each function takes clear inputs and returns a clear result. None of them print. Notice `summarise` guards against an empty list, returning an honest empty result rather than crashing. That guard is what a careful function looks like. If your Week 8 script also removed thousands separators or skipped `NA` values, add the same lines inside `clean_values` so the behaviour stays identical.

If you run the file now, nothing appears. That is correct: defining functions does not run them. The next step uses them.

Step 4: Wire them together

Below the functions, at the bottom of the file and not indented, add:

```
rows = read_rows("mydata.csv")
values = clean_values(rows, "area_km2")
result = summarise(values)
print(result)
```

Use your own file name and one of your column names. Read those four lines. They describe, in plain steps, exactly what the program does: read the rows, clean a column, summarise it, show the result. Someone can understand the whole program from these four lines without reading inside the functions. That readability is the entire point of refactoring.

Step 5: Confirm the output is identical

Run it:

```
uv run summary.py
```

Compare the output to what you recorded in Step 1. The count, smallest and largest should match exactly. The average is new, because your Week 8 script may not have calculated one. If the core numbers match, you refactored correctly: you improved the shape and kept the behaviour. If they differ, something changed; compare carefully and find the line responsible.

Step 6: The capstone, on your own data

Now bring the whole month together. Replace the four wiring lines from Step 4 with a function that summarises **several number columns** of your Month 1 dataset in one go, and the lines that use it:

```
def summarise_all(filename, columns):
    rows = read_rows(filename)
    report = {}
    for column in columns:
        values = clean_values(rows, column)
        report[column] = summarise(values)
    return report

report = summarise_all("mydata.csv", ["area_km2", "population"])

for column, stats in report.items():
    print(column, "->", stats)
```

Put `summarise_all` below the other three functions and above the lines that use it. Change the file name and the column names to real number columns in your data; use at least two, more if you have them.

This uses everything: reading, cleaning, functions, loops, conditions, dictionaries. `report` starts as an empty dictionary and gains one entry per column. `report.items()` then loops over it, giving you each column name and its statistics in turn. You should see one line per column, each followed by its count, smallest, largest and average.

This capstone reads your real Month 1 data and produces a full summary of it, in code you wrote and understand. That is a genuine, useful program, and it is the coding version of the data inspection you did by hand in Month 1.

Step 7: Hand in, and write your month reflection

Take the Week 9 screenshot and upload `summary.py`. Then add the short reflection described in the deliverable box to your README. Because this week is also the monthly task, check the whole repository against the list in End of Month 2 before you turn in.

You are done when

- Your inspection is split into small functions, each doing one job
- The functions return values; printing happens only where you call them
- The refactored output matches your Week 8 output for count, smallest and largest
- `summarise` guards against the empty case
- Your capstone summarises at least two real columns from your own Month 1 data
- You can read the wiring lines and explain what the program does
- Your Week 9 deliverable and monthly task are on GitHub and turned in

Independent challenge

Only if the practical felt comfortable. Not graded.

Find the extreme feature. Write a function that returns not just the largest value in a column, but which ward, borehole or place it belongs to. You will need to track the name alongside the value as you loop, updating both whenever you find a bigger value. This combines dictionaries, loops and conditions into something genuinely useful: the answer to "which place has the most of something", which is exactly the kind of question your Month 1 project asks.

Then, harder. Make your capstone write its results to a new CSV file rather than only printing them. Look up `csv.writer` in the Python documentation. You will loop over your report and write a row for each column. Writing files is the natural partner of reading them, and it means your script produces output another program, or QGIS, could open. That is the first hint of the pipelines you will build later in the year.

When it breaks

What you see	What it means	What to do
The output changed after refactoring	You altered behaviour, not just shape	Compare to your Step 1 record and find the line that differs
Running `summary.py` prints nothing	The functions are defined but never called	Add the wiring lines at the bottom, not indented
`NameError: name 'read_rows' is not defined`	You called a function above the line that defines it	Move all the `def` blocks above the lines that use them
`None` in the result	A function printed instead of returning	Check each function ends with `return`
One function is very long	It is doing more than one job	Split it further; aim for one job per function
`KeyError` after splitting into functions	A column name was passed wrong	Print the column name inside the function to see what it received
`report.items()` fails	`report` is not a dictionary	Check `summarise_all` builds and returns a dictionary

Check yourself

What is refactoring? Improving the shape of working code without changing what it does.

How do you know you refactored correctly? The output is identical before and after.

What is a dictionary? A collection that holds values by label, looked up by key rather than by position.

Why write small functions that each do one thing? They are easy to name, understand, test and fix, and the code that wires them together reads like a description of the program.

Your Week 9 deliverable and the monthly task

Upload to your Month 1 repository: `summary.py`, containing the three functions, `summarise_all`, and the lines that run it on your data. Then upload `week9.png` into the `screenshots` folder. You do not need to upload `notes.txt`.

The screenshot shows the VS Code terminal after `uv run summary.py`, with one line per column and its count, smallest, largest and average.

Add to your README: the Week 9 log line, then under it a heading `### Month 2 reflection` with three or four sentences of your own: which idea finally clicked this month, which is still fuzzy, and what your capstone tells you about your Month 1 data.

Submit on Google Classroom, on the Month 2 Week 9 task: your Month 1 repository link, `https://github.com/your-username/geoclean`, then click Turn in.

The Month 2 monthly task

The monthly task is your whole repository at the end of the month, graded as one piece. It passes when your Month 1 work is still there, untouched, all five weeks' files and all five screenshots are present with the exact names in the Deliverables table, the capstone runs on your own Month 1 data, and the README keeps its Month 1 content and has a log line for each Month 2 week plus the reflection.

Submit on Google Classroom, on the Month 2 monthly task: your Month 1 repository link, `https://github.com/your-username/geoclean`, then click Turn in. Yes, it is the same link as your weekly tasks and as Month 1. The monthly task looks at everything together.

End of Month 2

What you should have built

By the end of this month, your repository on GitHub, the same one you started in Month 1, should contain:

- all your Month 1 files, unchanged
- `README.md` with your Month 1 content, then a Month 2 section with a log line for each week and your reflection
- `hello.py`, proving your setup works (Week 5)
- `pyproject.toml` and `uv.lock`, the recipe for your environment, with pandas in it (Week 6)
- `check.py`, printing the pandas version (Week 6)
- `wards.csv` and `analyse.py`, your first program (Week 7)
- your data CSV and `inspect_data.py`, inspecting your own Month 1 data (Week 8)
- `summary.py`, refactored into clean functions, with a capstone that summarises your real data (Week 9)
- `screenshots/week5.png` to `screenshots/week9.png`, five screenshots
- no `.venv` folder

On your own computer you will also have a working Python, VS Code with the Python extension, and uv.

More importantly, you should be able to write a small program from nothing that reads a file, processes each row, and calculates a result, and understand every line of it. That is the real milestone.

For the standup

Three minutes. Cover four things.

Show your capstone running. Share your screen and run it on your own data. Watching your own code produce a summary of your real project is a genuine moment. Take it.

Explain one function. Pick one from your refactored script and walk through it line by line. If you can explain it, you understand it, and explaining it is how you find out.

What clicked, and what is still fuzzy. Name the idea that finally made sense this month, and the one still unclear. Somebody in your pod can help with the second. Your README reflection is a good script for this.

Where you got stuck. The error or the concept that cost you the most time. Everyone hit something; sharing yours helps the next person.

What happens in Month 3

Geospatial Python. You take everything from this month and point it at spatial data properly, using GeoPandas. Reading files, looping, functions that return, all of it, now working on real geographic data instead of a plain CSV.

This is where it starts to feel like GIS again, but written in code. Everything you clicked in Month 1, you will now write. The buffer, the spatial join, the reprojection, all of it becomes Python that runs the same way every time and can be run again on new data. Your project continues in the same repository, as it will every month until Month 12, now doing real spatial work.

A word about pace

This was the hardest month so far, by some distance. It is where GIS people become programmers, and that is a real transition, not a small one. If it felt hard, that is because it was hard, not because you are behind.

The test of readiness is simple. Can you write a small script that reads a file, does something to each row, and prints a result? If yes, you are ready for Month 3, even if half the details are still fuzzy. The details settle with use, and you will use all of them next month.

If you cannot do that yet, it is worth a few extra days before Month 3 rather than pushing on with a shaky foundation. The single best exercise is this: rebuild your Week

7 script from memory, without looking. Reading code is easy; writing it from nothing is what proves you understand it. Rebuild it, hit the errors, fix them, and it will stick.

Nobody is behind in week nine of a fifty-two week programme. This is the beginning.

The one idea to carry forward

If you take a single thing from this month above all others, take this, because it will save you more time than anything else: **print is for humans, return is for code.**

When a function does not seem to work. When a variable is mysteriously `None`. When a calculation vanishes into nothing. The cause is very often a missing `return`. A function that computes a value and only prints it has thrown that value away.

The moment something is unexpectedly empty, check for a missing `return` first. You will be right most of the time, and you will fix in seconds what would otherwise cost you an hour.

For pod leads

Month 2 is where a cohort thins if it is going to, because this is where GIS people meet real programming for the first time. Watch for it, and intervene early and kindly.

Week 5. Installation and setup problems peak here, along with the first time handing in code through GitHub. Anybody silent by Thursday probably has a broken setup and feels embarrassed about it. Message them directly, not in the group, and point them to the setup guide and video. A broken install is not a broken person; it is a different first task. Before the Week 5 deadline, open each pod member's repository link in a private browser window. A private repository or a mistyped link is the most common reason good work gets marked missing, and it takes one minute to catch. Watch too for members who create a brand new repository for Month 2. Point them back to their Month 1 repository: the project is one repository for the whole year.

Week 6. The idea of environments is abstract, and some members will not see the point of it. That is fine. They do not need to love it, they need the three commands running. Focus on `uv init`, `uv add`, `uv run` working, not on deep understanding, which comes with use. Watch for anyone trying to upload their whole folder to GitHub and failing on the `.venv` folder.

Week 7. The big one. This is where people either cross into programming or stall. Encourage members to post the actual error text, not "it doesn't work"; an error message is answerable in one reply, a vague complaint is not. Celebrate first working scripts loudly. A member who prints their first calculation has done something real, and should hear so.

Week 8. The print-versus-return confusion will fill your pod chat. This is good; it means people are writing functions. Point them at the diagram and the rule: when a variable is empty, look for a missing `return`. Expect data problems too: invisible markers at the start of CSVs, thousands separators, and `NA` values. The Week 8 When it breaks table covers each one.

Week 9. Refactoring week is calmer, with no new material. Use it to help anyone who fell behind catch up, and to check that every member's repository has all five weeks' files and screenshots before the monthly task is turned in. Encourage the confident members to help the struggling ones, because explaining code is how the confident ones consolidate what they have learned.

Throughout. You are not the answer desk. When you do not know something, say so and pass it up. Most questions this month can be answered by another member, which is exactly the peer dynamic the programme is built on.

Glossary

Every term used in this pack, in plain language. Come back to this whenever something is unfamiliar.

append: adding an item to the end of a list, with `wards.append("Enugu")`.

argument: a value you pass into a function when you call it. In `km2_to_hectares(30.8)`, the argument is `30.8`.

boolean: a type with only two values, `True` and `False`. For yes-or-no facts.

call: to run a function, by writing its name with brackets: `km2_to_hectares(30.8)`.

code editor: a program for writing code, such as VS Code.

command: an instruction you type into the terminal for it to run.

comment: a note in code, starting with `#`, that Python ignores.

commit: a saved change to a repository, with a short message describing it.

comprehension: a compact way to build a list from another list in one line.

condition: code that chooses between actions depending on whether a test is true, using `if`, `elif` and `else`.

continue: inside a loop, skip the rest of the current turn and move to the next item.

CSV: comma-separated values, a simple text format for tables, with commas between values.

debugging: finding out why code does not do what you expected, usually by printing values to see what is really happening.

dictionary: a collection that holds values by label, looked up by key: `ward["name"]`.

directory: another word for a folder.

encoding: the rule a file uses to store letters as numbers. This pack uses `utf-8-sig` for CSVs.

environment: a private box of installed packages belonging to one project, so projects do not interfere with each other. uv keeps it in a hidden folder called `.venv`.

extension: an add-on for VS Code. The Python extension makes VS Code understand Python.

float: a number with a decimal point, like `12.4`. For measurements and areas.

function: a named, reusable recipe that takes inputs and returns a result, defined with `def`.

GitHub: a website that stores repositories online, so your work can be shared and graded.

import: bringing a package or module into your code so you can use it: `import pandas`.

indentation: the spaces at the start of a line, which tell Python what is inside a loop, a function or a condition. Not decoration; it changes meaning.

index: the position of an item in a list, counted from zero. `wards[0]` uses index 0.

integer: a whole number, with no decimal point, like `774`. For counts and years.

JSON: a text format for structured data, which Python reads into dictionaries and lists.

key: the label used to look up a value in a dictionary. In `ward["name"]`, the key is `"name"`.

list: a collection of values in order, made with square brackets: `["Agodi", "Bodija"]`.

loop: code that repeats an action for each item in a list, using `for`.

module: a file of ready-made Python code you can import, such as `csv` or `json`.

None: a special value meaning “nothing”. What a function returns if it has no `return`.

package: code somebody else wrote and shared, which you install and import, like pandas.

path: the full address of a file or folder, such as `C:\Users\Ada\dev\geoclean`.

PATH: the list of folders your computer searches when you type a command name. Different from a path, despite the name.

print: show a value on the screen for a person to read.

prompt: the symbol in the terminal, like `>` or `$`, where you type your command.

public repository: a repository anyone with the link can open. Yours must be public so it can be graded.

pyproject.toml: the readable file describing your project and the packages it needs.

README: the front page of a repository, `README.md`, telling readers what the project is.

refactoring: improving the shape of working code without changing what it does.

repository (repo): a project folder stored on GitHub, with its own web link.

return: hand a value back out of a function so the rest of your code can use it.

slice: part of a list taken by position, such as `lines[1:]`, everything from position one onwards.

string: text, written inside quotes, like `"Ibadan"`.

terminal: the window where you type commands for the computer to run. PowerShell on Windows, Terminal on Mac.

type: the kind of a value: string, integer, float or boolean. Decides what you can do with it.

uv: the tool this programme uses to manage projects, environments and packages.

uv.lock: the file uv writes recording the exact version of every package, for reproducibility.

variable: a labelled box that holds a value, created with `state = "Oyo"`.

VS Code: the code editor you write and run code in.

Month 2 of 12. GeoDev Lab Africa, Cohort One.

Learn. Build. Collaborate. Transform.